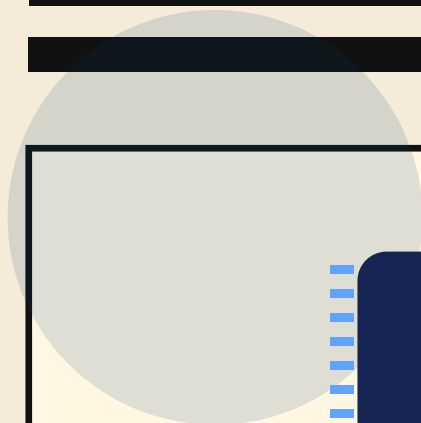
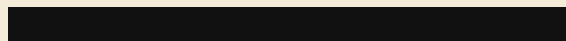
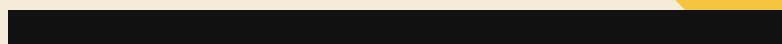
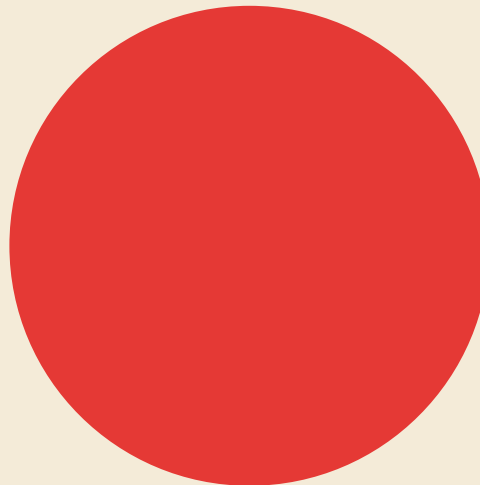




KernelESP

**A Beginner's Guide to a Tiny UNIX
for the ESP8266**

Version 0.10.0 - Bauhaus navigation edition

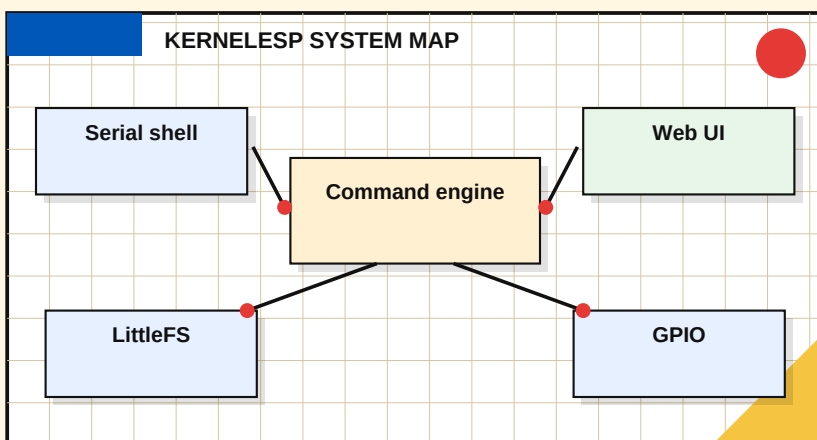
Serial shell - Web UI - Automation - Hardware - Recovery - Real projects

KernelESP Beginner's Book

This book is written for people who are new to microcontrollers, new to embedded web interfaces, or simply want a calm path into KernelESP. It treats the ESP8266 as a tiny UNIX-like machine: it has commands, files, logs, scripts, cron jobs, relays, sensors, inputs, mail alerts and a web interface that keeps everything visible.

How to read it

Start at Part 1 if the board is new to you. Jump to the projects in Part 8 when you want irrigation, alarms or scheduled actions. Keep Part 10 nearby when something goes wrong.





■ About this edition

This generated edition is part of the KernelESP repository. It is not a replacement for the command reference; it is a guided tour that explains why each feature exists, when to use it and how to test it safely on real hardware.

The examples prefer 24-hour time, non-blocking timers and explicit recovery paths. That style matters on a small ESP8266: clarity is a safety feature, and short commands keep the serial console and web UI responsive.

Try it

```
help
health
free
wifi status
crontab -l
relay status
```

Table of Contents

This index is clickable in PDF viewers that support internal links. Use it like a control panel for the book.

■ Part 1: Orientation

— Meet the Tiny UNIX	JUMP
— Serial Console First Contact	JUMP
— Find the Web Interface	JUMP
— Use the HTTP API	JUMP
— Read System Health	JUMP

■ Part 2: Files and Shell Confidence

— Understand LittleFS	JUMP
— Create and Inspect Files	JUMP
— Use Pipes Like a Pro	JUMP
— History and Aliases	JUMP
— Dry Run Mode	JUMP

■ Part 3: Networking and Time

— Scan and Connect Wi-Fi	JUMP
— Save Wi-Fi Profiles	JUMP
— Recover Wi-Fi	JUMP
— Static IP or DHCP	JUMP
— Fallback Access Point	JUMP
— Set the Hostname	JUMP
— Read the Clock	JUMP
— Use NTP Kick	JUMP
— Schedule NTP Twice Daily	JUMP
— Manual Time Rescue	JUMP

■ Part 4: Hardware Control

— Relay Basics	JUMP
— Name Relays Well	JUMP
— Pulse a Relay	JUMP
— Timers Instead of Sleep	JUMP
— Repeating Timers	JUMP

■ Part 5: Scheduling and Sensors

— Cron Daily Jobs	JUMP
— Cron Weekly Jobs	JUMP
— Cron and Safety	JUMP
— Sensors First Read	JUMP
— BME280 and BMP280	JUMP
— Sensor Rules	JUMP
— Hysteresis	JUMP

■ Part 6: Automation Language

— Scenes	JUMP
— Persistent State	JUMP
— Digital Inputs	JUMP
— Input Events	JUMP
— C-like Expressions	JUMP
— Blocks and Else	JUMP
— Variables with let	JUMP
— Constants with define	JUMP
— Functions	JUMP
— Scripts	JUMP
— Boot Script	JUMP



■ Part 7: Web, Mail and Diagnostics

— Logs	JUMP
— Backups	JUMP
— Mail Setup	JUMP
— Mail Alerts	JUMP
— Diagnostics	JUMP
— Live UI	JUMP
— Command Runner	JUMP
— Script Editor	JUMP
— Local Help	JUMP
— Security Basics	JUMP
— Web Lockout	JUMP

■ Part 8: Garden Projects

— GPIO Choices	JUMP
— External Relay Power	JUMP
— Long Cable Runs	JUMP
— Outdoor Enclosures	JUMP
— Solenoid Valves	JUMP
— Flow Measurement	JUMP
— Build Zone One	JUMP
— Add Zone Two	JUMP
— Rain Skip Pattern	JUMP
— Temperature Window Pattern	JUMP
— Daily Health Report	JUMP

■ Part 9: Recovery and Release

— Recover from Bad Automation	JUMP
— Serial Rescue Workflow	JUMP
— Safe Boot Thinking	JUMP
— Firmware Upload	JUMP
— Asset Upload	JUMP
— Release Testing	JUMP

■ Part 10: Going Further

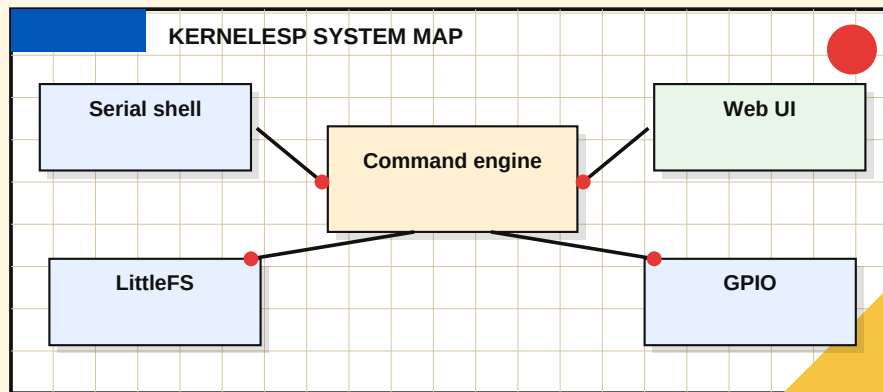
— Command Reference Habits	JUMP
— API Integrations	JUMP
— Build a Morning Irrigation Program	JUMP
— Build a Temperature Fan Program	JUMP
— Build an Offline Logger	JUMP
— Keep the System Fast	JUMP
— Know the Limits	JUMP
— Document Your Installation	JUMP
— Final Beginner Checklist	JUMP

■ Part 11: Command Atlas

■ Final Notes

The Learning Map

Every chapter pair follows the same rhythm: first a plain-English explanation, then a lab page with commands, checks and a safe next step.



- Inspect before changing: health, logs and status commands are your first tools.
- Prefer non-blocking actions: timers and cron are safer than long sleeps.
- Make names human: trees, flowers and fan are better than relay1, relay2 and gpio5.
- Keep recovery nearby: serial, backups and disarm are part of the design.

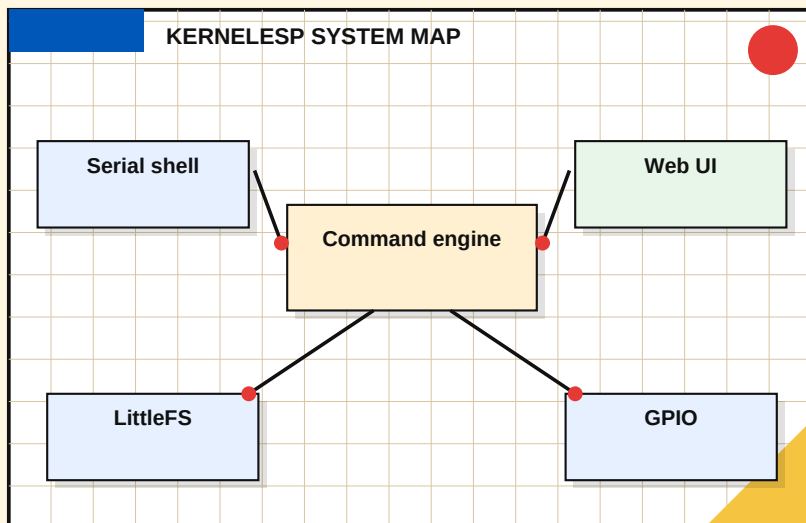
PART 1

Orientation

This part covers Meet the Tiny UNIX, Serial Console First Contact, Find the Web Interface and builds toward practical confidence.

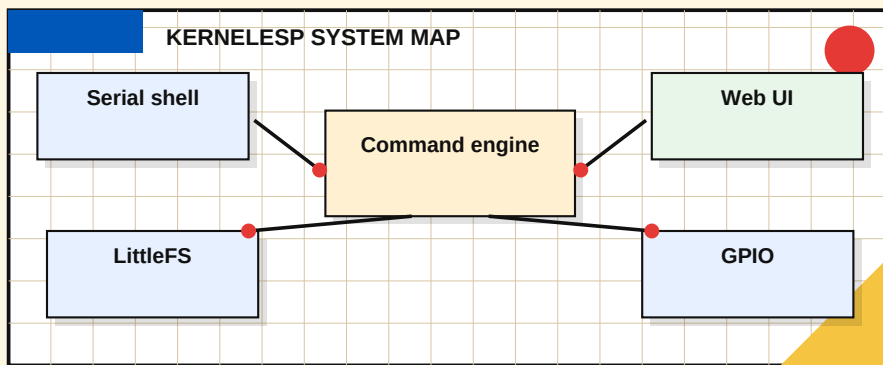
Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.



1. Meet the Tiny UNIX

KernelESP turns an ESP8266 into a small networked computer with a shell, files, web pages and automation.



■ The mental model

Think of the board as a tiny server. It cannot run Linux, but it borrows the useful habits: inspect first, change second, save the result.

Why this matters

A mental model prevents panic. If it has commands, files and logs, you can reason about it.

▲ Common beginner mistake

Treating the ESP8266 like a black box. KernelESP is inspectable; use that gift.

Lab 1: Meet the Tiny UNIX

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
uname
version
whoami
help
health
```

■ What to check

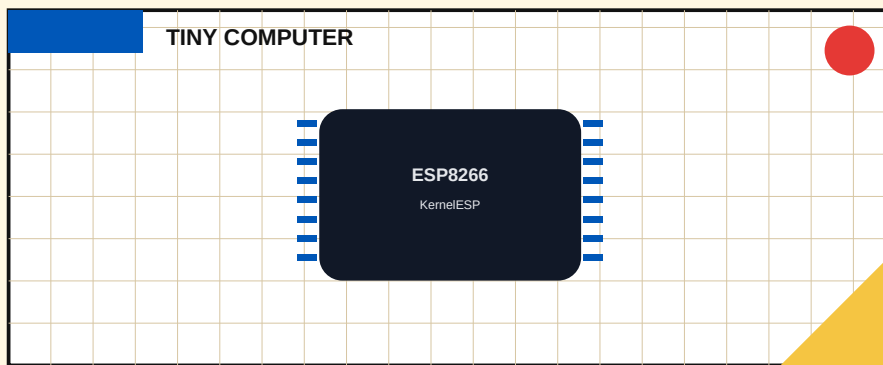
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Run help for three commands you do not understand yet and write down what each one controls.

2. Serial Console First Contact

The serial console is the most reliable way to talk to the board, especially during Wi-Fi trouble.



■ The mental model

Serial is the local keyboard and screen. It works before the network is ready and remains the best rescue route.

Why this matters

Every serious installation should keep a serial rescue path available during setup.

▲ Common beginner mistake

Changing Wi-Fi and then closing the only working console.

Lab 2: Serial Console First Contact

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
help
wifi status
free
df
dmesg
```

What to check

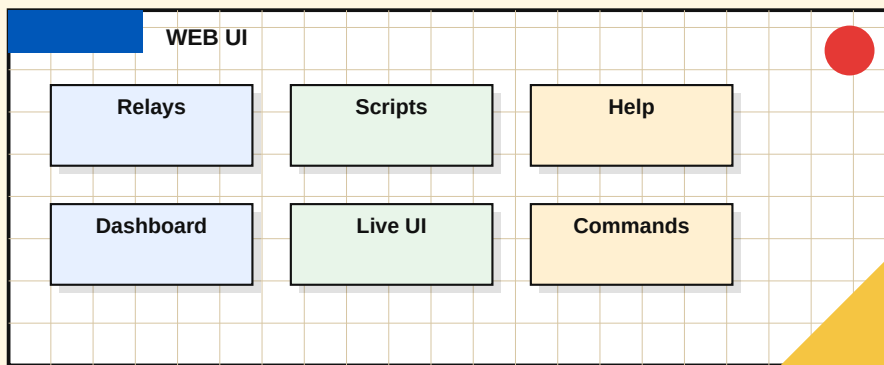
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Unplug Wi-Fi mentally: which commands would still help you recover?

3. Find the Web Interface

The web UI gives beginners a friendly control panel for commands, relays, scripts, logs and help.



■ The mental model

The browser is a remote window into the same command engine used by serial.

Why this matters

Seeing the same state in two places builds confidence that you are controlling the right board.

▲ Common beginner mistake

Assuming the browser is broken when the board simply changed IP.

Lab 3: Find the Web Interface

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
wifi ip  
wifi status  
hostname  
health
```

What to check

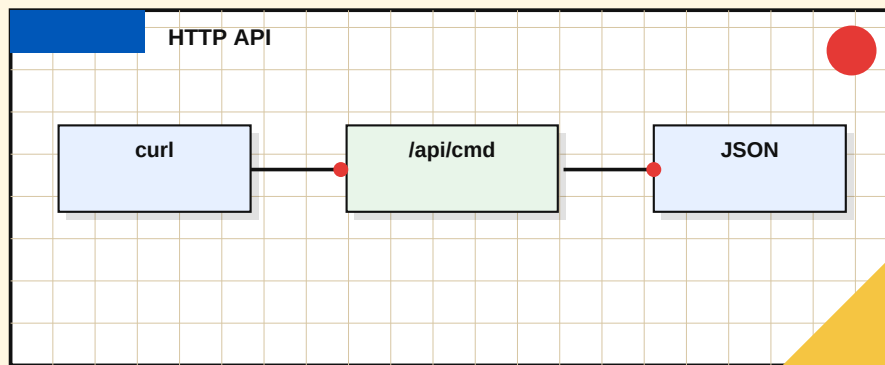
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Open the IP address in a browser and compare Dashboard data with health output.

4. Use the HTTP API

The API lets scripts and other computers ask Kerne!ESP to run commands and return JSON.



■ The mental model

The API is the command line with a URL around it. It is useful for integrations and tests.

Why this matters

APIs are how small devices join bigger systems without needing a cloud service.

▲ Common beginner mistake

Forgetting URL encoding when a command contains spaces, quotes or symbols.

Lab 4: Use the HTTP API

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
curl -G http://<ip>/api/cmd --data-urlencode key=admin --data-urlencode c=free  
curl http://<ip>/api/status?key=admin
```

What to check

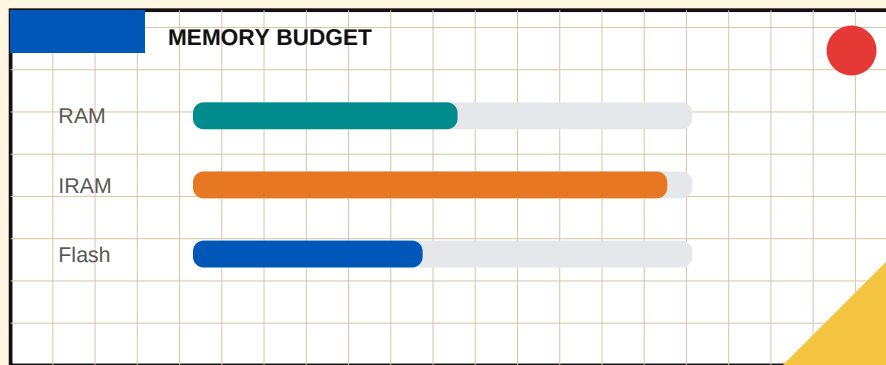
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Run an API command from your laptop and compare it with the web command runner.

5. Read System Health

Health commands tell you whether the board is stable before you add automation.



■ The mental model

Heap, Wi-Fi state, filesystem space and time are vital signs.

Why this matters

A baseline makes future debugging factual instead of emotional.

▲ Common beginner mistake

Ignoring low heap or high fragmentation until the web UI feels slow.

Lab 5: Read System Health

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
health
free
uptime
df
sysinfo
```

What to check

- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Record heap before and after opening Live UI, then after running a script.

PART 2

Files and Shell Confidence

This part covers Understand LittleFS, Create and Inspect Files, Use Pipes Like a Pro and builds toward practical confidence.

Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.

LITTLEFS LAYOUT

/	
/etc	config, cron, relays
/home	scripts and notes
/www	web interface
/var/log	kernel.log
/func	persistent functions

6. Understand LittleFS

KernelESP stores configuration, scripts and web assets in LittleFS.

LITTLEFS LAYOUT	
/	
/etc	config, cron, relays
/home	scripts and notes
/www	web interface
/var/log	kernel.log
/func	persistent functions

■ The mental model

LittleFS is the board's small disk. It survives reboot but has limited space.

Why this matters

Files make automations persistent and inspectable.

▲ Common beginner mistake

Saving everything in one giant file instead of small named files.

Lab 6: Understand LittleFS

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
ls /  
ls /etc  
ls /home  
df  
du /
```

What to check

- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Create a tiny note in /home, read it, then delete it.

7. Create and Inspect Files

The shell can create, append, read, search and remove files.

LITTLEFS LAYOUT	
/	
/etc	config, cron, relays
/home	scripts and notes
/www	web interface
/var/log	kernel.log
/func	persistent functions

■ The mental model

File commands are small building blocks for scripts and backups.

Why this matters

A beginner who can inspect files can recover from many configuration mistakes.

▲ Common beginner mistake

Editing a file without reading it first.

Lab 7: Create and Inspect Files

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
write /home/hello.txt hello
append /home/hello.txt world
cat /home/hello.txt
grep hello /home/hello.txt
rm /home/hello.txt
```

■ What to check

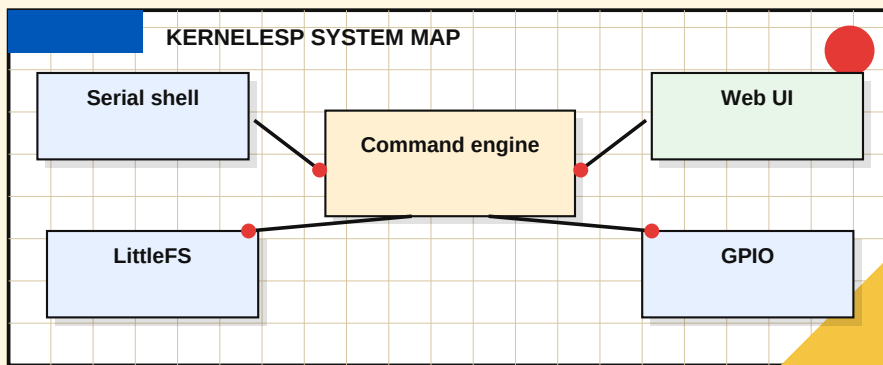
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Create a checklist file for your installation.

8. Use Pipes Like a Pro

Pipes connect small commands so one command filters another.



■ The mental model

A pipe is a little conveyor belt: left command produces text, right command narrows it down.

Why this matters

Pipes make the tiny shell feel surprisingly UNIX-like.

▲ Common beginner mistake

Using pipes for everything; sometimes one direct command is clearer.

Lab 8: Use Pipes Like a Pro

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
health | grep wifi  
dmesg | tail -n 5  
cat /var/log/kernel.log | grep cron  
health | wc -l
```

What to check

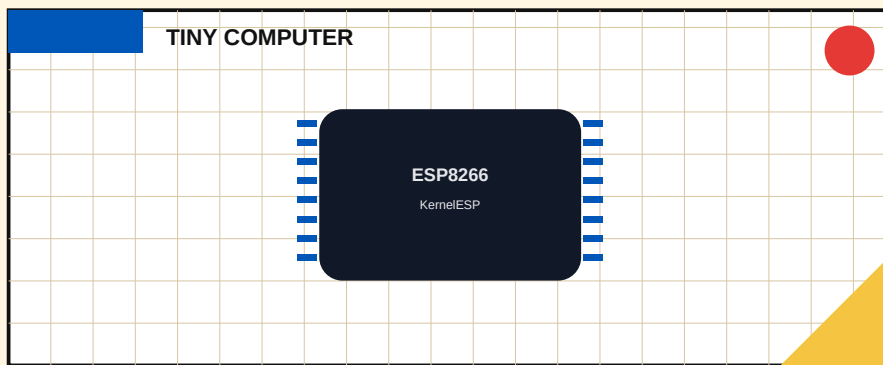
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Find all log lines containing wifi or ntp.

9. History and Aliases

History and aliases save typing during repeated setup work.



■ The mental model

An alias is a short nickname for a longer command.

Why this matters

Shortcuts reduce mistakes when commands are long.

▲ Common beginner mistake

Making aliases with names that hide real commands.

Lab 9: History and Aliases

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
history
alias ll ls /
ll
alias save
history save
```

What to check

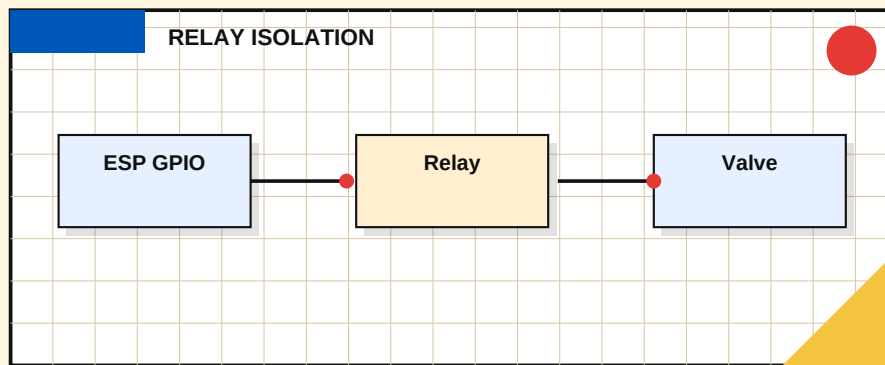
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Create an alias for health and remove it later.

10. Dry Run Mode

Dry run helps test dangerous commands without energizing hardware.



■ The mental model

It is a rehearsal mode for actuators.

Why this matters

Safety-first habits matter more than clever automation.

▲ Common beginner mistake

Testing new relay logic directly on live hardware.

Lab 10: Dry Run Mode

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
dryrun on
relay on valve1
relay status
dryrun off
```

What to check

- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Use dryrun before connecting a pump, valve or mains relay.

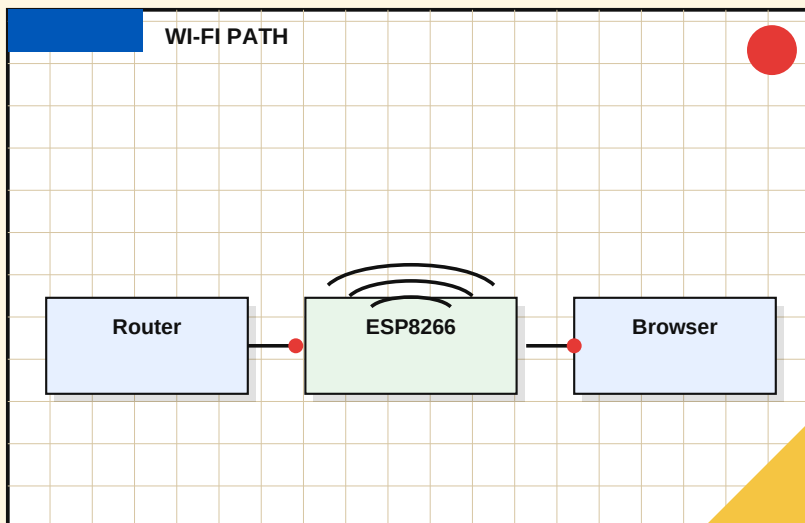
PART 3

Networking and Time

This part covers Scan and Connect Wi-Fi, Save Wi-Fi Profiles, Recover Wi-Fi and builds toward practical confidence.

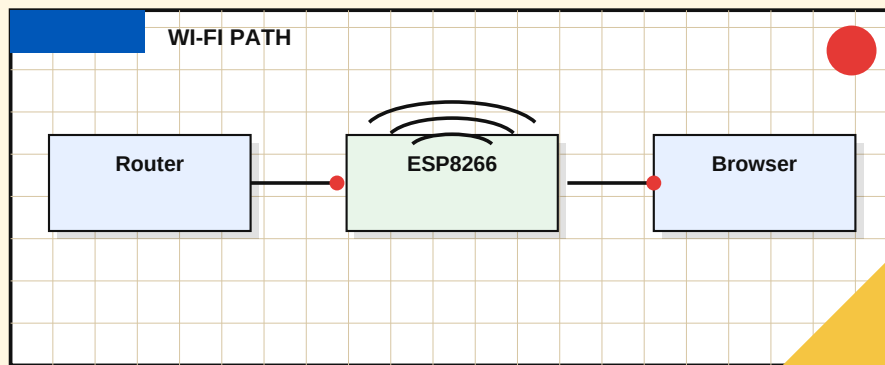
Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.



11. Scan and Connect Wi-Fi

Wi-Fi commands connect the board while keeping the shell responsive.



■ The mental model

Connection starts in the background; you keep control while it negotiates.

Why this matters

Non-blocking Wi-Fi avoids freezing the rescue console.

▲ Common beginner mistake

Running many network commands while the first connection is still settling.

Lab 11: Scan and Connect Wi-Fi

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
wifi scan
wifi connect MySSID MyPassword
wifi status
wifi wait 30
```

What to check

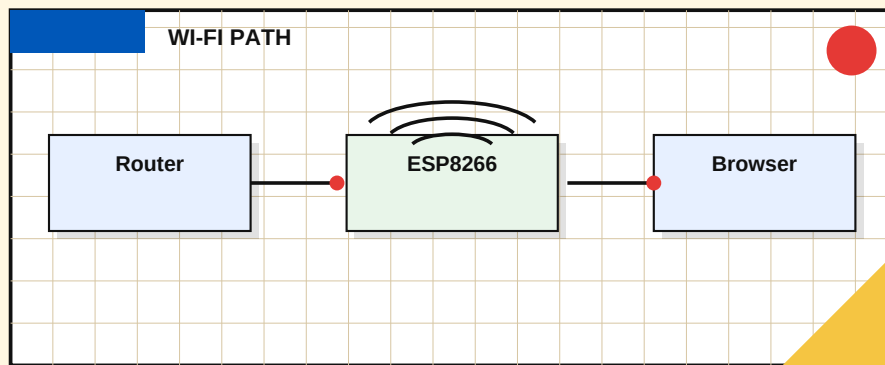
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Connect, wait, and verify IP address without rebooting.

12. Save Wi-Fi Profiles

Profiles make it easy to move between networks or recover from credential changes.



■ The mental model

A profile is a named Wi-Fi memory.

Why this matters

Installations in gardens and workshops often move during testing.

▲ Common beginner mistake

Forgetting that saved credentials persist after reboot.

Lab 12: Save Wi-Fi Profiles

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
wifi save HomeNetwork SecretPassword  
wifi profile list  
wifi autoconnect on  
wifi reconnect
```

What to check

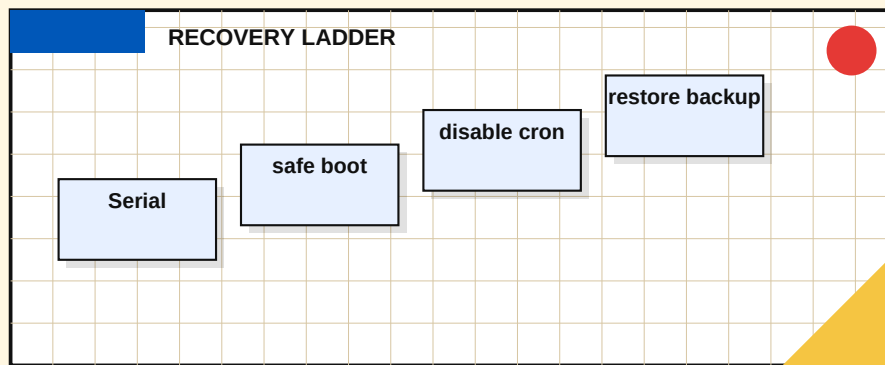
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Save a profile and inspect it before relying on it.

13. Recover Wi-Fi

Recovery commands help when the board joins the wrong network or gets confused.



■ The mental model

Wi-Fi recovery is a ladder: inspect, reconnect, forget, recover, SDK reset only when needed.

Why this matters

A calm recovery path prevents unnecessary reflashing.

▲ Common beginner mistake

Using sdkreset as the first move instead of the last careful step.

Lab 13: Recover Wi-Fi

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
wifi diag  
wifi recover  
wifi disconnect  
wifi reconnect  
wifi sdkreset --yes
```

What to check

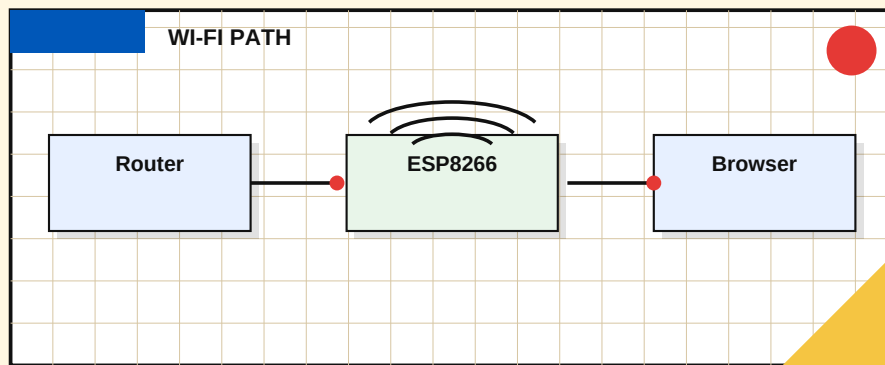
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Write a recovery checklist before you need it.

14. Static IP or DHCP

KernelESP can use DHCP or a static address depending on your installation.



■ The mental model

DHCP is convenient; static IP is predictable.

Why this matters

Predictable addressing makes web access and API scripts easier.

▲ Common beginner mistake

Setting a static IP outside the router subnet.

Lab 14: Static IP or DHCP

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
wifi dhcp on reconnect  
wifi static 192.168.1.50 192.168.1.1 255.255.255.0 8.8.8.8 1.1.1.1 reconnect  
wifi net
```

What to check

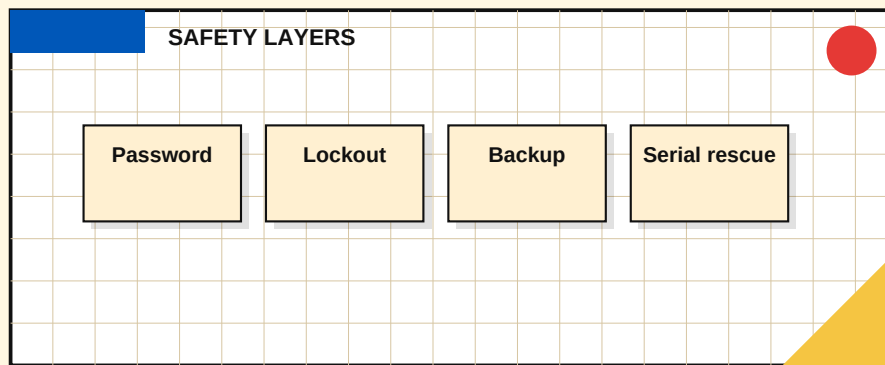
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Decide which mode is safer for your garden controller.

15. Fallback Access Point

The fallback AP can expose setup access when normal Wi-Fi fails.



■ The mental model

AP mode is a temporary lifeboat, not the normal harbor.

Why this matters

Remote installations need a way back in after router changes.

▲ Common beginner mistake

Leaving setup access open without thinking about physical security.

Lab 15: Fallback Access Point

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
ap status
config get fallback.ap
config set fallback.ap on
ap start
```

What to check

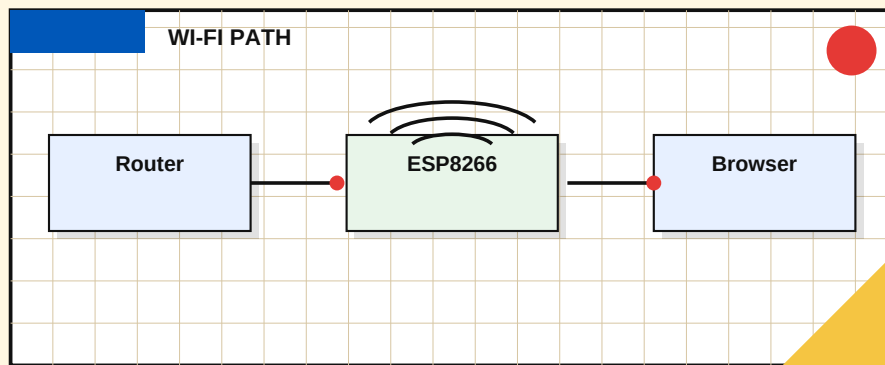
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Confirm whether fallback AP is enabled on your board.

16. Set the Hostname

A good hostname makes the board easier to find and recognize.



■ The mental model

Names beat numbers when you have more than one controller.

Why this matters

Clear names prevent editing the wrong controller.

▲ Common beginner mistake

Using the same hostname for multiple boards.

Lab 16: Set the Hostname

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
hostname
hostname kernalesp-garden
wifi reconnect
wifi status
```

What to check

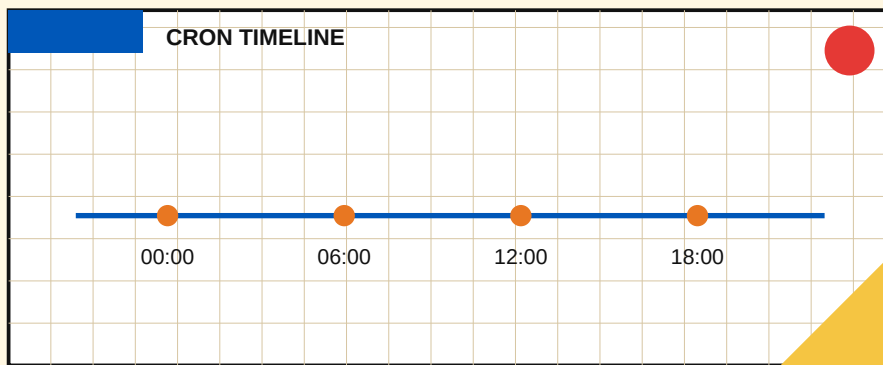
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Choose a name that describes the physical location.

17. Read the Clock

Time unlocks cron, logs and safe irrigation windows.



■ The mental model

The clock is the board's calendar. Without it, schedules cannot mean much.

Why this matters

Every schedule depends on trustworthy time.

▲ Common beginner mistake

Assuming cron is wrong when NTP never synced.

Lab 17: Read the Clock

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
date
date -u
ntp status
time status
```

■ What to check

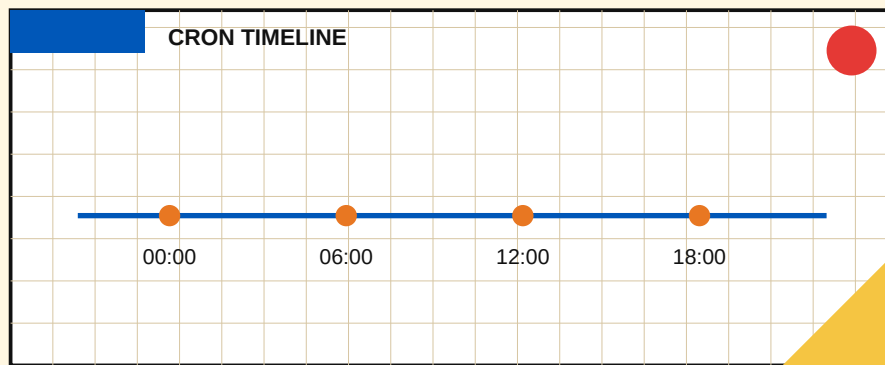
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Compare local time and UTC time.

18. Use NTP Kick

ntp kick starts time synchronization without blocking the rest of the system.



■ The mental model

It is a polite request: start syncing, then give the shell back.

Why this matters

Non-blocking time sync keeps automations responsive.

▲ Common beginner mistake

Expecting instant perfect time on a weak Wi-Fi network.

Lab 18: Use NTP Kick

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
ntp kick
ntp status
date
dmesg | tail -n 5
```

What to check

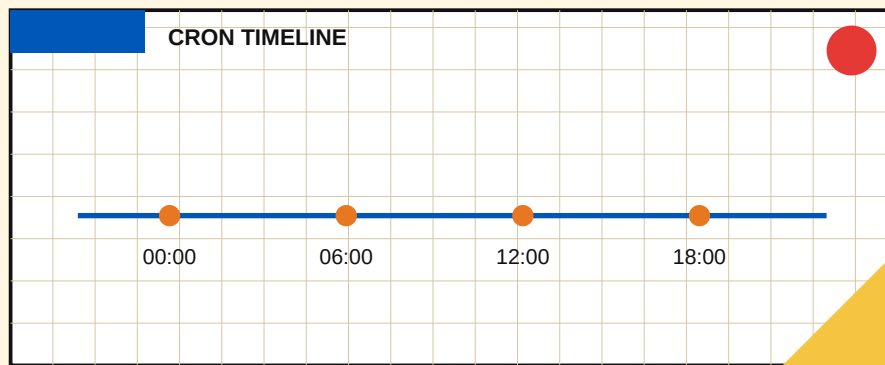
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Run `ntp kick`, then keep using the shell while time settles.

19. Schedule NTP Twice Daily

A simple cron pair keeps time fresh without hammering NTP servers.



■ The mental model

Most garden controllers do not need minute-by-minute clock correction.

Why this matters

Twelve-hour sync is a practical balance for ESP8266 projects.

▲ Common beginner mistake

Scheduling NTP every minute because more feels safer.

Lab 19: Schedule NTP Twice Daily

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
cron add daily 00:00 ntp kick  
cron add daily 12:00 ntp kick  
crontab -l
```

What to check

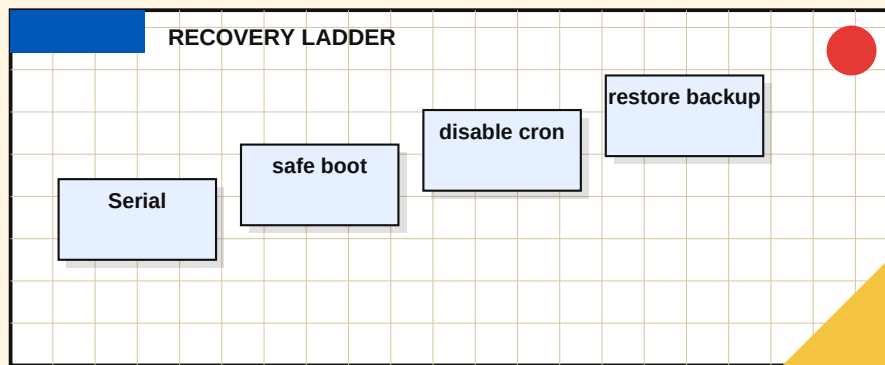
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Confirm both jobs appear in cron.

20. Manual Time Rescue

Manual date setting is useful when Wi-Fi is down and you still need schedules.



■ The mental model

Manual time is a temporary crutch until NTP returns.

Why this matters

Time rescue can keep logs understandable during network outages.

▲ Common beginner mistake

Forgetting to return to NTP after manual testing.

Lab 20: Manual Time Rescue

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
date
date set 2026-04-29 08:30:00
date
ntp kick
```

What to check

- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Practice manual time on a bench board before field deployment.

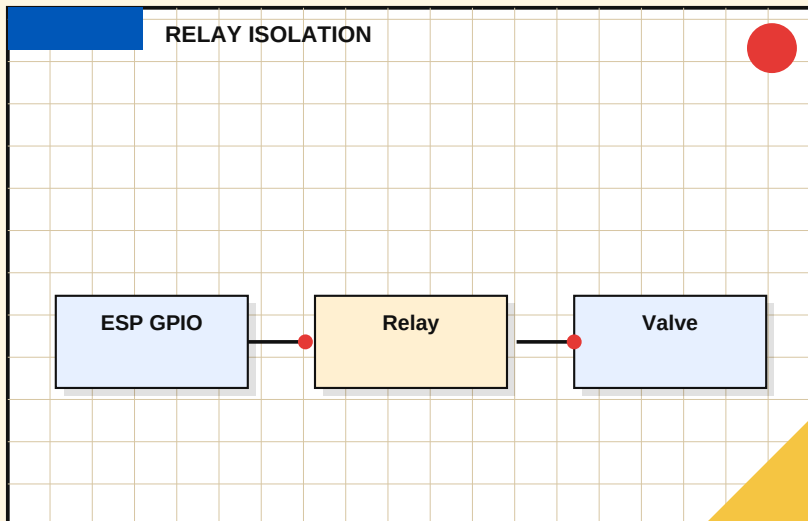
PART 4

Hardware Control

This part covers Relay Basics, Name Relays Well, Pulse a Relay and builds toward practical confidence.

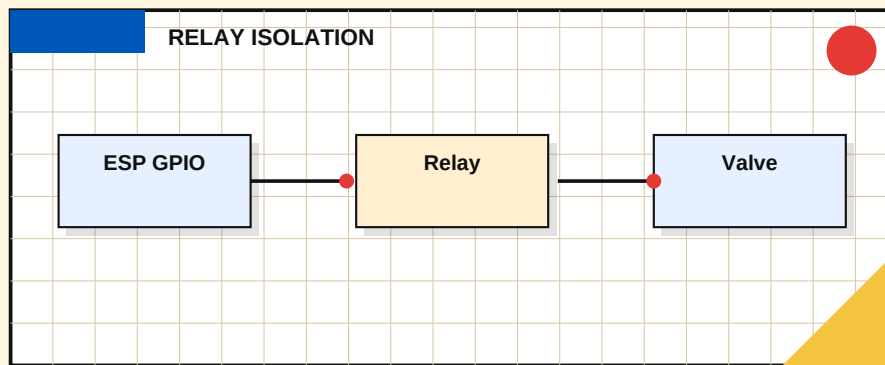
Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.



21. Relay Basics

Relays let a 3.3 V logic pin control a separate load safely when wired correctly.



■ The mental model

The ESP gives a signal; the relay handles the heavier electrical side.

Why this matters

Relays are the bridge between software and physical action.

▲ Common beginner mistake

Powering relay coils from the ESP 3.3 V pin.

Lab 21: Relay Basics

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
relay add valve1 D1 active_low
relay status
relay on valve1
relay off valve1
```

What to check

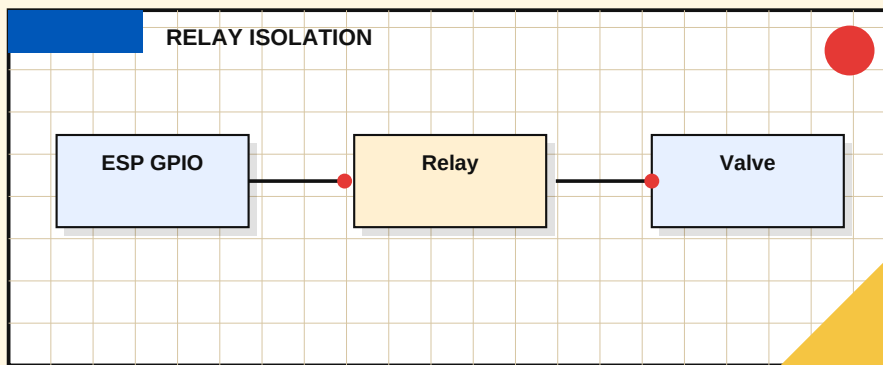
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Use dryrun until the relay module behavior is known.

22. Name Relays Well

Good relay names make scripts readable and safer.



■ The mental model

Names are labels for real-world equipment.

Why this matters

Readable names reduce mistakes months later.

▲ Common beginner mistake

Calling everything relay1, relay2 and forgetting which is which.

Lab 22: Name Relays Well

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
relay add trees D1 active_low
relay add flowers D2 active_low
relay status
relay rm flowers
```

What to check

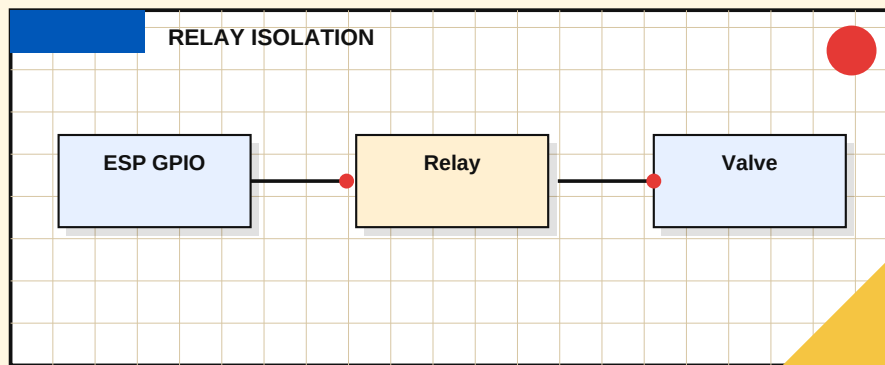
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Choose names based on the thing controlled, not the GPIO number.

23. Pulse a Relay

A pulse turns something on briefly and then turns it off automatically.



■ The mental model

Pulse is a short controlled action, useful for tests or momentary loads.

Why this matters

Short actions are easier to validate than long schedules.

▲ Common beginner mistake

Using pulse for long irrigation when a timer would be clearer.

Lab 23: Pulse a Relay

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
relay pulse valve1 500
relay status
log | tail -n 5
```

What to check

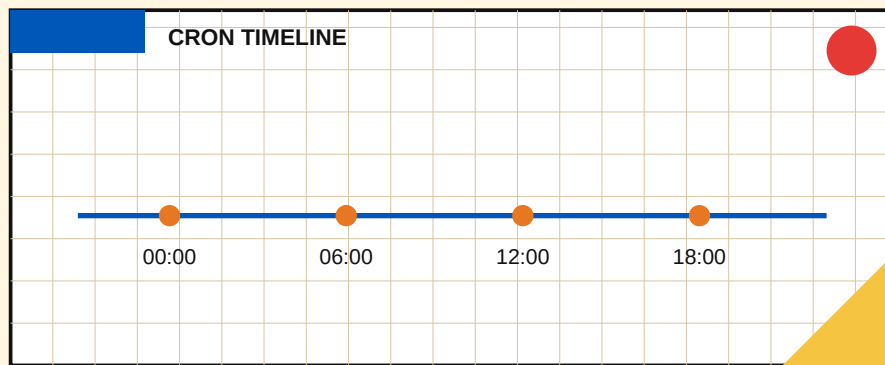
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Pulse a test LED before pulsing a real valve.

24. Timers Instead of Sleep

Timers schedule future actions without blocking the shell.



■ The mental model

A timer is a reminder the system keeps while you do other things.

Why this matters

Non-blocking timers are essential for reliable automation.

▲ Common beginner mistake

Using sleep for long actuator runs.

Lab 24: Timers Instead of Sleep

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
relay on valve1
timer once 600000 relay off valve1
timer list
relay status
```

What to check

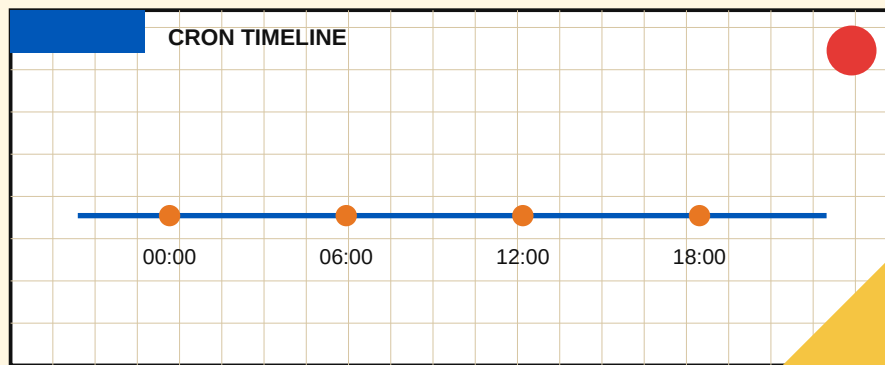
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Start a short 10 second test timer and watch it complete.

25. Repeating Timers

Repeating timers run commands at intervals while the system stays responsive.



■ The mental model

They are useful for lightweight periodic maintenance.

Why this matters

Timers are simple when the interval matters more than the clock time.

▲ Common beginner mistake

Leaving experimental timers running after tests.

Lab 25: Repeating Timers

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
timer every 60000 health
timer list
timer clear
```

What to check

- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Create a short repeating timer, observe it, then clear it.

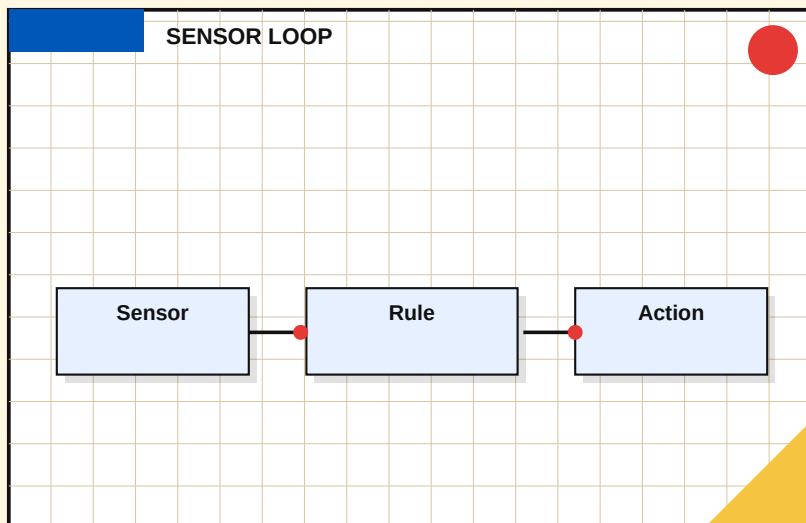
PART 5

Scheduling and Sensors

This part covers Cron Daily Jobs, Cron Weekly Jobs, Cron and Safety and builds toward practical confidence.

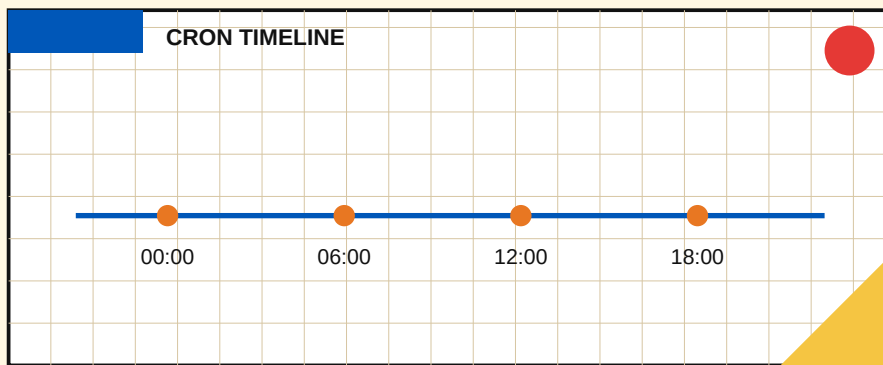
Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.



26. Cron Daily Jobs

Cron runs commands at human clock times.



■ The mental model

Cron is a calendar-based scheduler.

Why this matters

Daily cron jobs are perfect for irrigation windows.

▲ Common beginner mistake

Adding on jobs without matching off or safety timers.

Lab 26: Cron Daily Jobs

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
cron add daily 06:00 relay on trees
cron add daily 06:10 relay off trees
crontab -l
```

What to check

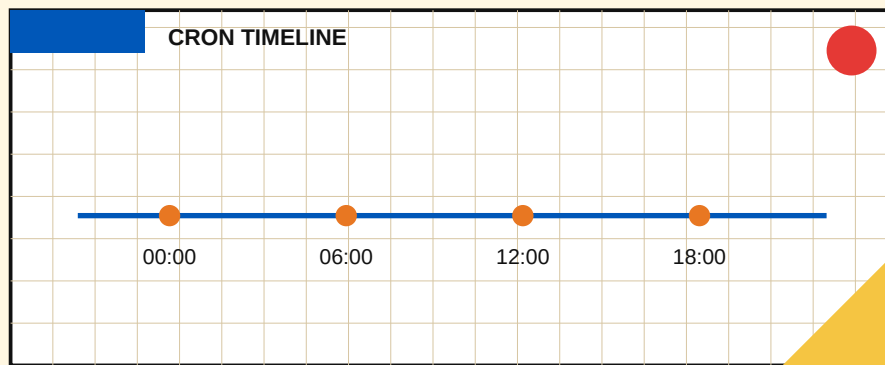
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Create a harmless logger job and remove it.

27. Cron Weekly Jobs

Weekly schedules support maintenance and weekly routines.



■ The mental model

The clock decides when; the command engine decides what.

Why this matters

Weekly patterns keep maintenance visible.

▲ Common beginner mistake

Forgetting local time and using AM/PM habits in a 24-hour system.

Lab 27: Cron Weekly Jobs

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
cron add weekly mon 08:00 logger monday_check
cron add weekly sun 22:30 sh /home/weekly.sh
crontab -l
```

What to check

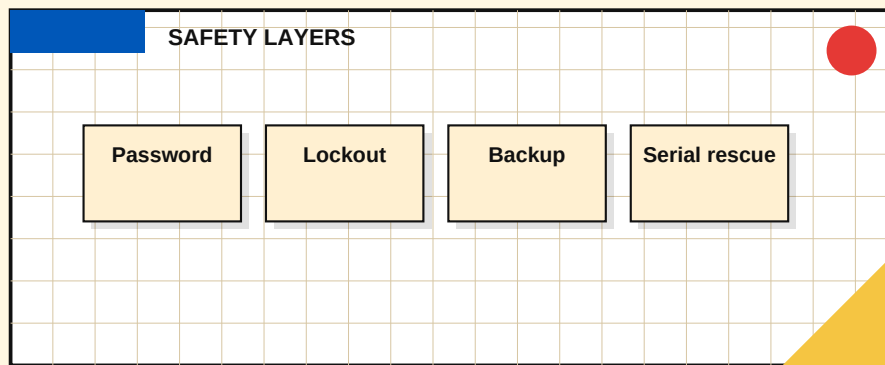
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Schedule a weekly health log entry.

28. Cron and Safety

Cron jobs should be short, inspectable and reversible.



■ The mental model

A schedule is only safe if the action is safe.

Why this matters

A pause switch is a gift to your future self.

▲ Common beginner mistake

Debugging a live schedule without disarming it first.

Lab 28: Cron and Safety

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
crontab -l
cron rm <id>
armed
disarm
arm
```

What to check

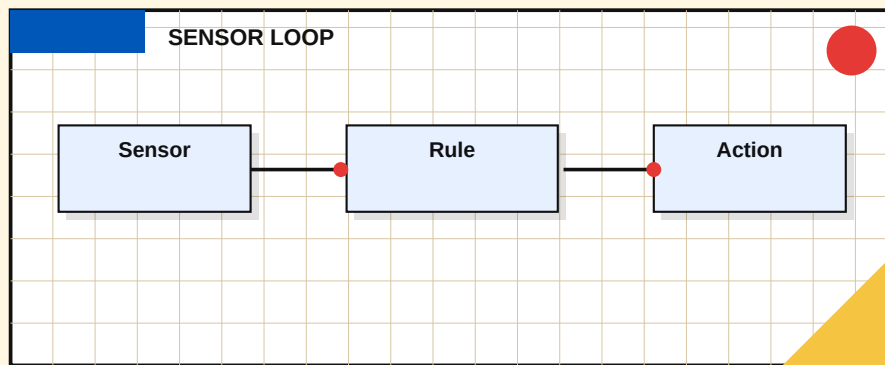
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Find the quickest way to pause automations.

29. Sensors First Read

Sensors provide temperature, humidity and pressure values for decisions.



■ The mental model

Sensor values are measurements, not commands. Read before you react.

Why this matters

Rules and automations are only as good as the sensor data.

▲ Common beginner mistake

Writing rules before confirming the sensor works.

Lab 29: Sensors First Read

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
sensor begin
sensor read
sensor status
health
```

■ What to check

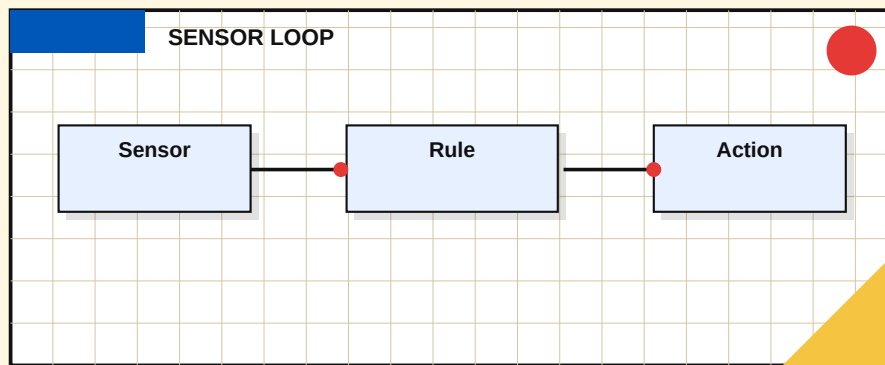
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Compare sensor output before and after touching the sensor gently.

30. BME280 and BMP280

Environmental sensors can share I2C wiring when connected correctly.



■ The mental model

I2C is a small data bus: power, ground, clock and data.

Why this matters

Physical wiring errors often look like software bugs.

▲ Common beginner mistake

Mixing 5 V sensor modules with 3.3 V-only GPIO without checking.

Lab 30: BME280 and BMP280

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
i2c scan
sensor begin
sensor read
dmesg | tail -n 10
```

What to check

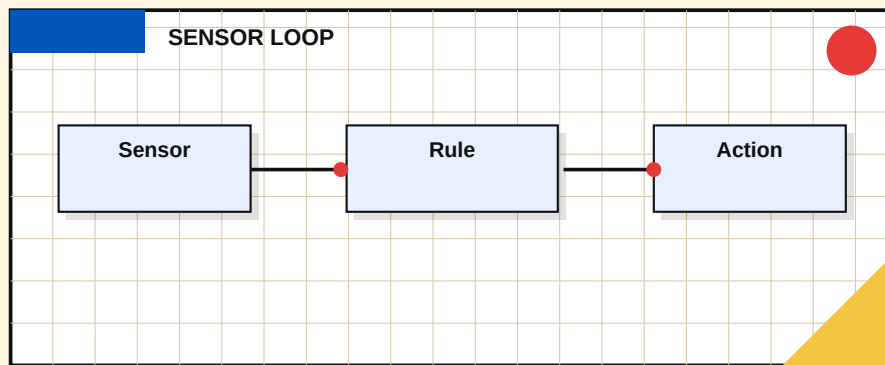
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Run i2c scan and write down detected addresses.

31. Sensor Rules

Rules turn measurements into actions.



■ The mental model

A rule is a small thermostat-like decision.

Why this matters

Rules are simpler than scripts for direct threshold behavior.

▲ Common beginner mistake

Using one threshold for both on and off, causing rapid toggling.

Lab 31: Sensor Rules

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
rule add temp gt 40 relay fan on
rule list
rule rm <id>
relay status
```

■ What to check

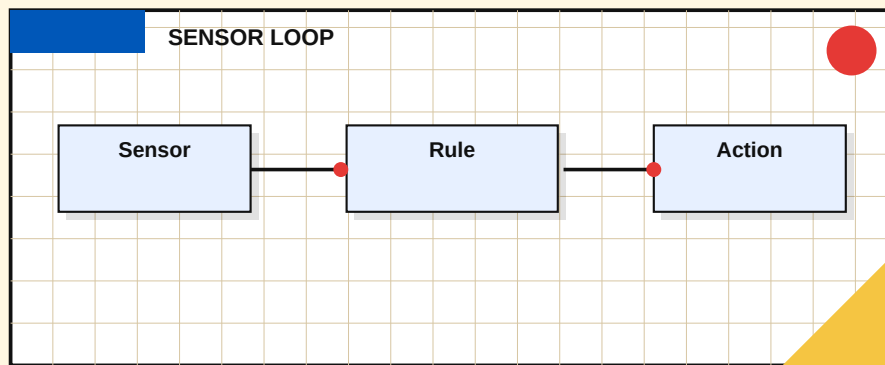
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Create a logger-only rule before controlling a relay.

32. Hysteresis

Hysteresis prevents relays from chattering around a threshold.



■ The mental model

Use one threshold to turn on and another to turn off.

Why this matters

Mechanical relays and valves should not toggle every few seconds.

▲ Common beginner mistake

Setting on at 40 and off at 39.9 in noisy conditions.

Lab 32: Hysteresis

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
if (temp >= 40) relay on fan
if (temp <= 35) relay off fan
rule list
```

What to check

- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Choose a gap wide enough for your real environment.

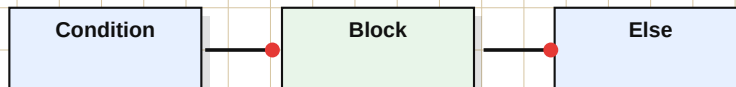
PART 6

Automation Language

This part covers Scenes, Persistent State, Digital Inputs and builds toward practical confidence.

Beginner promise

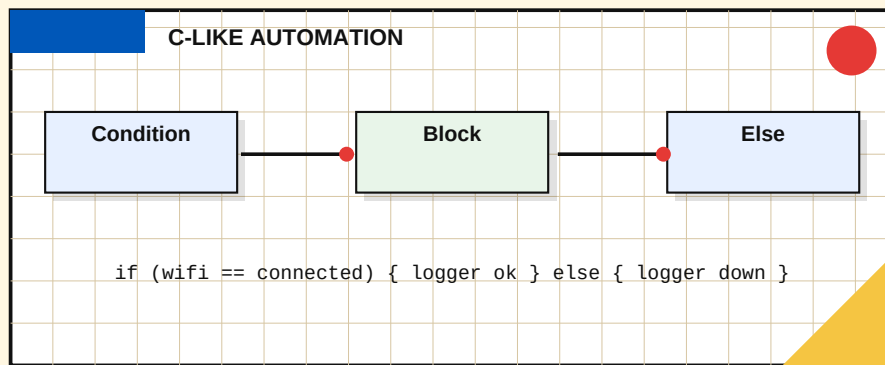
You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.

C-LIKE AUTOMATION

```
if (wifi == connected) { logger ok } else { logger down }
```

33. Scenes

Scenes group commands into named actions.



■ The mental model

A scene is a macro for a state you want often.

Why this matters

Scenes make web buttons and scripts easier to understand.

▲ Common beginner mistake

Putting too much logic inside a scene; use scripts for logic.

Lab 33: Scenes

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
scene add night relay off lights; logger night_scene
scene run night
scene list
scene show night
```

What to check

- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Create a scene that only logs, then inspect it.

34. Persistent State

State stores small values that survive reboot.

LITTLEFS LAYOUT	
/	
/etc	config, cron, relays
/home	scripts and notes
/www	web interface
/var/log	kernel.log
/func	persistent functions

■ The mental model

State is the board's notebook.

Why this matters

Persistent state lets automations remember modes.

▲ Common beginner mistake

Using state for large data logs instead of small key values.

Lab 34: Persistent State

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
state set irrigation.enabled 1
state get irrigation.enabled
state list
state rm irrigation.enabled
```

What to check

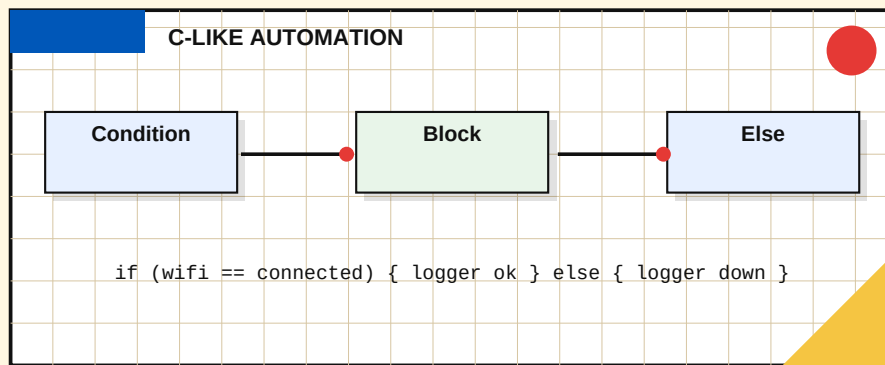
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Store a harmless value and confirm it persists after reboot later.

35. Digital Inputs

Inputs react to buttons, switches and sensor pulses.



■ The mental model

An input is the board listening to a pin.

Why this matters

Names make input automations readable.

▲ Common beginner mistake

Forgetting pullup behavior and reading inverted logic.

Lab 35: Digital Inputs

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
input add rain D2 pullup 50
input list
input rm rain
```

What to check

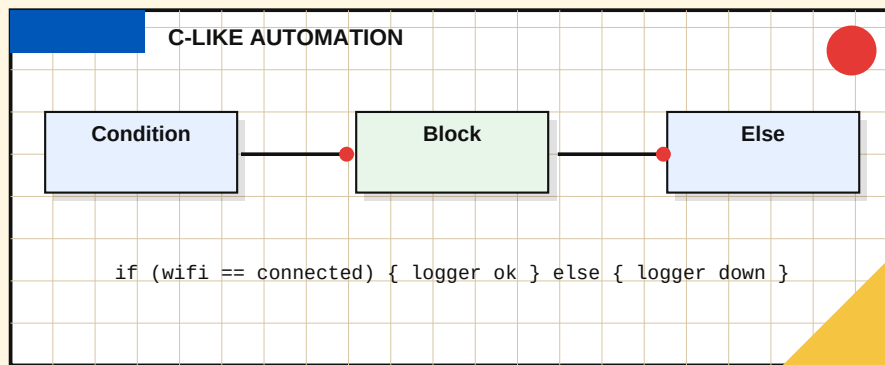
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Add a named input even before wiring the final sensor.

36. Input Events

Input events attach commands to high, low or change transitions.



■ The mental model

The event is the trigger; the command is the response.

Why this matters

Logging input events proves your wiring before actuation.

▲ Common beginner mistake

Starting a pump from an untested bouncing switch.

Lab 36: Input Events

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
input add button D7 pullup 50
input on button low logger button_pressed
input list
```

What to check

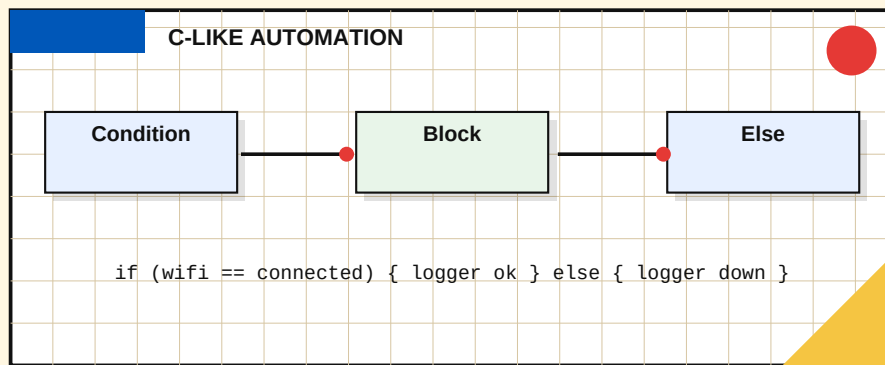
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Use logger first, hardware second.

37. C-like Expressions

The automation language supports familiar comparisons and logic.



■ The mental model

Expressions answer yes or no before running a command.

Why this matters

Readable conditions make complex projects approachable.

▲ Common beginner mistake

Expecting full C. This is a tiny expression layer, not a compiler.

Lab 37: C-like Expressions

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
if (wifi == connected) logger online
if (armed == on && wifi == connected) logger ready
if (!(armed == on)) logger paused
```

What to check

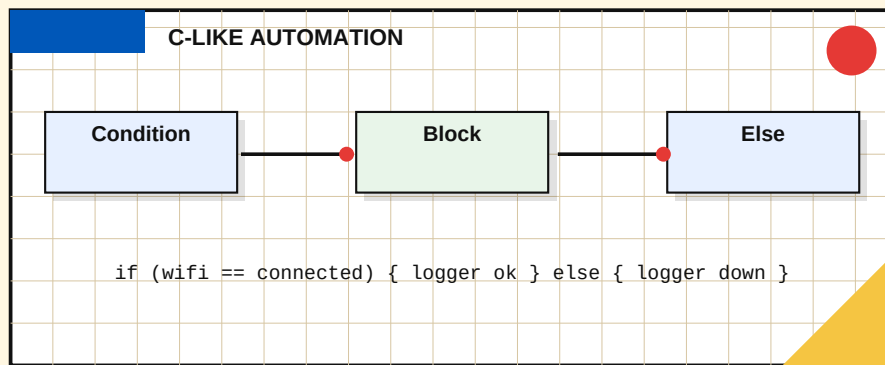
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Change armed state and observe which condition fires.

38. Blocks and Else

Blocks let one condition run several commands.



■ The mental model

A block is a small group of commands separated by semicolons.

Why this matters

Blocks reduce duplicated conditions.

▲ Common beginner mistake

Writing multiline if blocks in scripts; KernelESP rejects them for stability.

Lab 38: Blocks and Else

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
if (wifi == connected) { logger online; mail health "online" } else { logger offline }  
if (armed == on) { relay status; timer list }
```

What to check

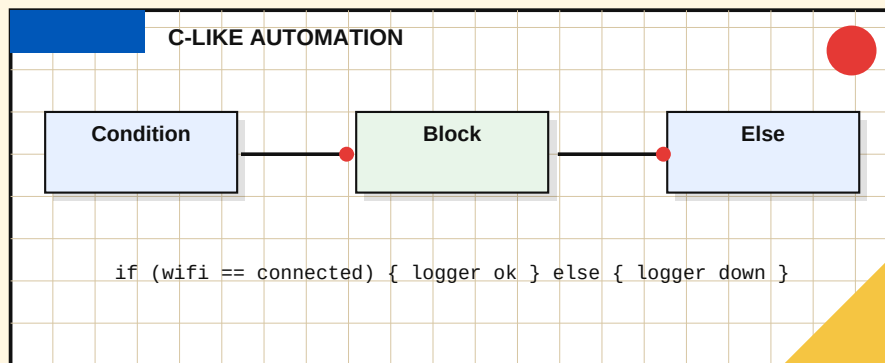
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Keep blocks on one line in scripts.

39. Variables with let

let creates persistent variables for automation decisions.



■ The mental model

Variables are readable knobs for behavior.

Why this matters

Variables let beginners tune behavior without rewriting every command.

▲ Common beginner mistake

Forgetting that variable names are case-insensitive.

Lab 39: Variables with let

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
let irrigation.enabled = 1
let max.temp = 35
let list
if (irrigation.enabled == 1 && temp < max.temp) logger allowed
```

What to check

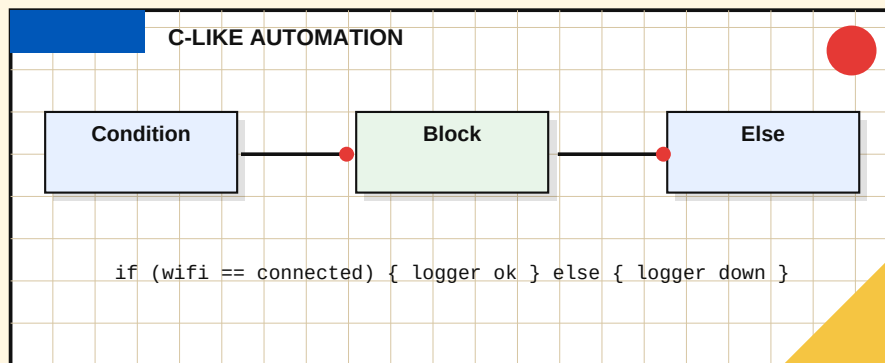
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Create a variable for your preferred watering duration.

40. Constants with define

define creates named constants used in expressions.



■ The mental model

Constants are labels for numbers and times.

Why this matters

Names explain intent better than bare numbers.

▲ Common beginner mistake

Assuming HOT and hot are different constants.

Lab 40: Constants with define

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
define HOT 40
define MORNING_END 10:00
define list
if (temp >= HOT && time < MORNING_END) logger hot_morning
```

What to check

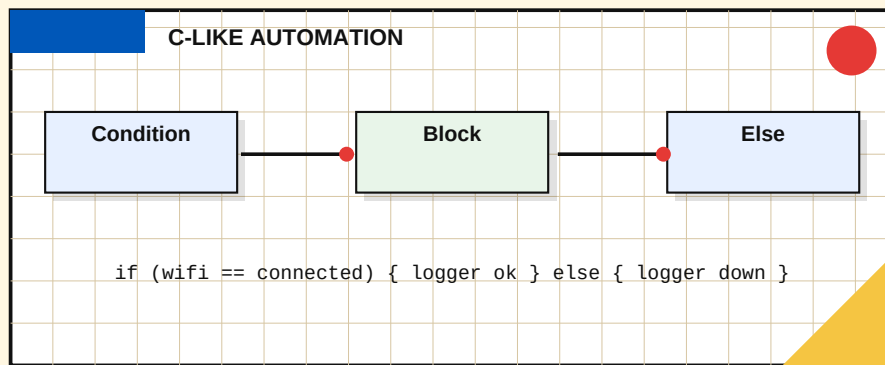
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Replace a magic number in one script with a constant.

41. Functions

Functions store named command blocks under /func.



■ The mental model

A function is a reusable mini script.

Why this matters

Functions keep cron and input actions short.

▲ Common beginner mistake

Calling functions from inside if branches; use direct commands in branches.

Lab 41: Functions

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
function water_zone1 { relay on valve1; timer once 600000 relay off valve1; logger zone1_starte
function list
function show water_zone1
call water_zone1
```

What to check

- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Create a function that only logs before using relays.

42. Scripts

Scripts are files containing commands, one per line.

LITTLEFS LAYOUT	
/	
/etc	config, cron, relays
/home	scripts and notes
/www	web interface
/var/log	kernel.log
/func	persistent functions

■ The mental model

A script is a recipe stored on the board.

Why this matters

Scripts make setup repeatable.

▲ Common beginner mistake

Saving untested scripts into boot.

Lab 42: Scripts

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
write /home/test.sh echo start
append /home/test.sh health
sh -n /home/test.sh
sh /home/test.sh
```

What to check

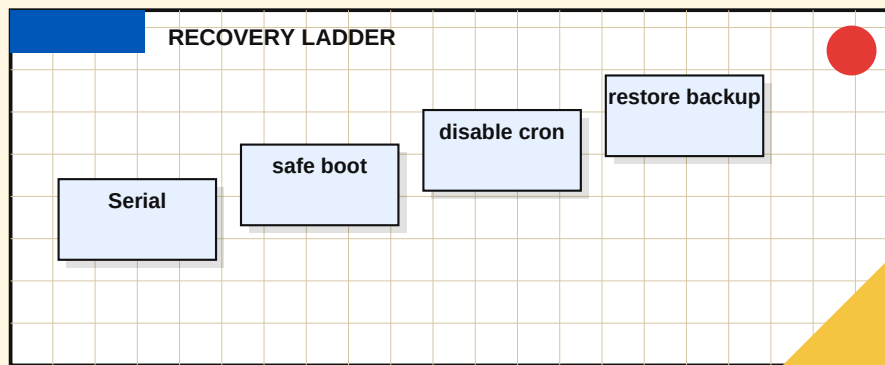
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Validate scripts with `sh -n` before running them.

43. Boot Script

The boot script runs startup commands.



■ The mental model

Boot is the board's morning routine.

Why this matters

A safe boot script makes power loss recovery boring.

▲ Common beginner mistake

Putting long sleeps or risky relay actions at boot.

Lab 43: Boot Script

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
boot show
boot set /etc/boot.sh
cat /etc/boot.sh
onboot list
```

■ What to check

- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Keep boot commands short and non-blocking.

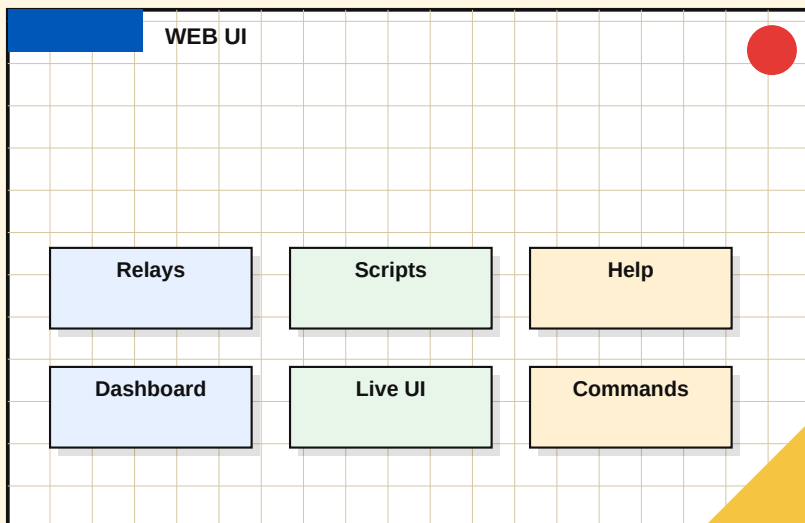
PART 7

Web, Mail and Diagnostics

This part covers Logs, Backups, Mail Setup and builds toward practical confidence.

Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.



44. Logs

Logs tell the story of what happened.

LITTLEFS LAYOUT	
/	
/etc	config, cron, relays
/home	scripts and notes
/www	web interface
/var/log	kernel.log
/func	persistent functions

■ The mental model

When confused, read the diary.

Why this matters

Logs turn mystery failures into timelines.

▲ Common beginner mistake

Clearing logs before copying the useful clue.

Lab 44: Logs

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
log
tail -n 20 /var/log/kernel.log
grep cron /var/log/kernel.log
dmesg
```

What to check

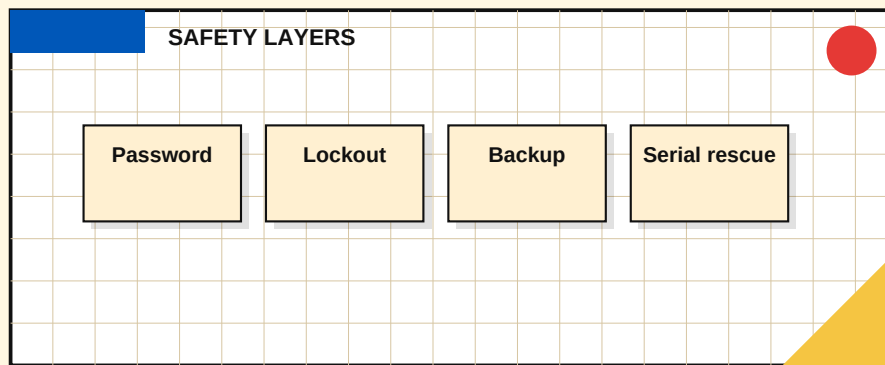
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Find the last Wi-Fi-related log entry.

45. Backups

Backups preserve configuration before experiments.



■ The mental model

A backup is a snapshot you can return to.

Why this matters

Small boards are easier to experiment with when rollback exists.

▲ Common beginner mistake

Testing risky automation without a current backup.

Lab 45: Backups

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
backup
export relays
cat /etc/config.txt
ls /etc
```

What to check

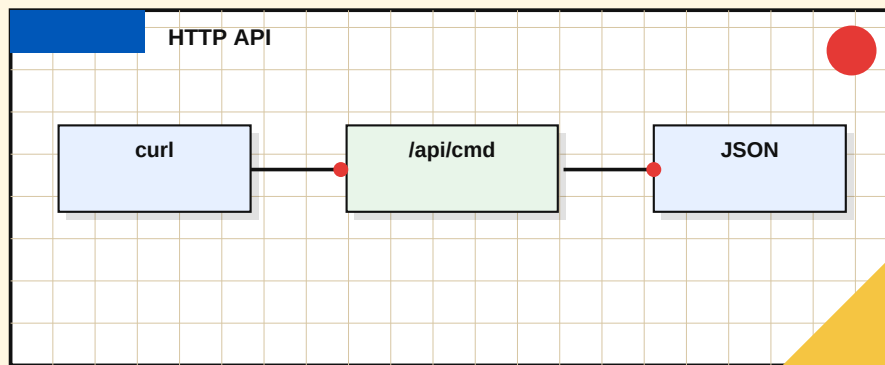
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Make a backup before changing Wi-Fi or cron.

46. Mail Setup

Mail alerts let the board report important events.



■ The mental model

Email is an outbound notification path.

Why this matters

Notifications are most useful when they are boringly reliable.

▲ Common beginner mistake

Triggering emails every few seconds from a noisy rule.

Lab 46: Mail Setup

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
mail status
mail config
mail test
mail health "KernelESP health"
```

What to check

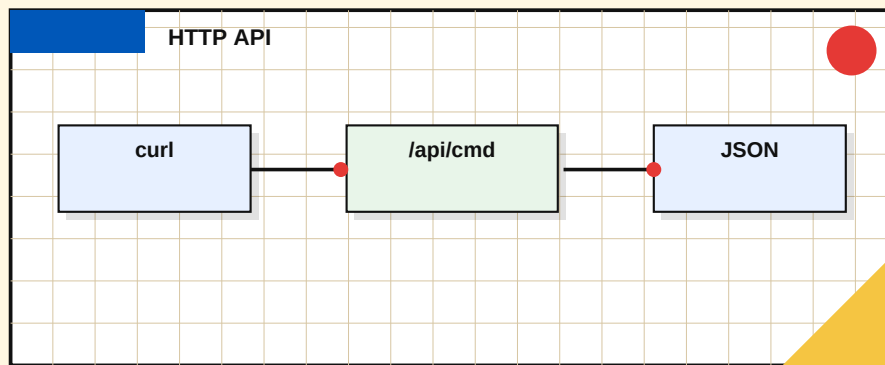
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Send a test email before using mail in automation.

47. Mail Alerts

A good alert says what happened and what to do next.



■ The mental model

Alerts are for humans, not logs with wings.

Why this matters

Useful alerts make remote installs manageable.

▲ Common beginner mistake

Sending alert storms during unstable Wi-Fi.

Lab 47: Mail Alerts

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
if (wifi == connected) mail health "daily health"
logger daily_health_sent
mail status
```

What to check

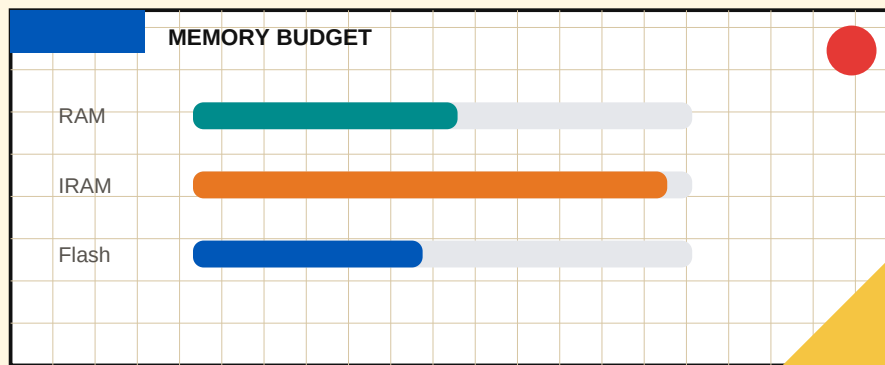
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Create one daily health email, not ten noisy alerts.

48. Diagnostics

Diagnostics bundle the facts needed for support.



■ The mental model

A diagnostic is a snapshot of the system state.

Why this matters

Good diagnostics reduce guesswork.

▲ Common beginner mistake

Only reporting symptoms without command output.

Lab 48: Diagnostics

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
diag
board
health
sysinfo
wifi diag
```

What to check

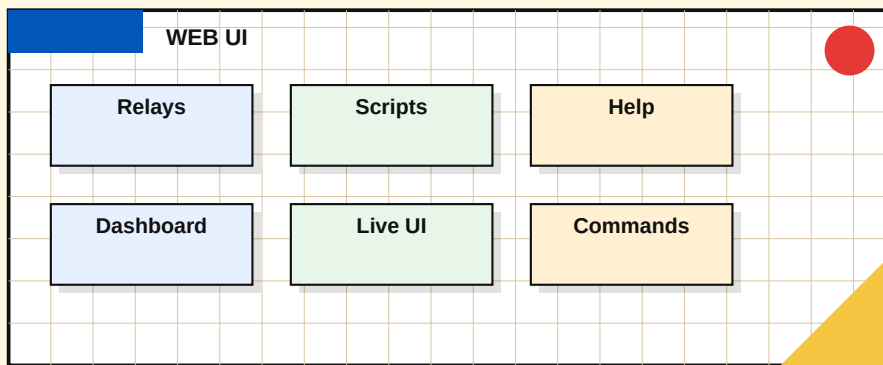
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Run diag before and after a major change.

49. Live UI

Live UI shows changing system state in the browser.



■ The mental model

It is a dashboard, not a replacement for careful command checks.

Why this matters

Refreshing less often keeps the ESP responsive.

▲ Common beginner mistake

Leaving many browser tabs polling the board during critical automation.

Lab 49: Live UI

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
health
free
timer list
crontab -l
```

What to check

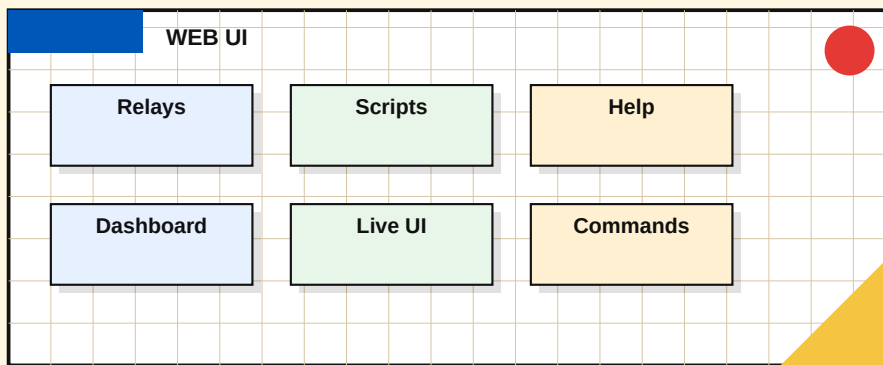
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Open Live UI and watch heap while running harmless commands.

50. Command Runner

The web command runner is a beginner-friendly shell.



■ The mental model

It is ideal for controlled experiments from a browser.

Why this matters

Seeing identical behavior builds trust in the model.

▲ Common beginner mistake

Pasting a long unreviewed command from notes.

Lab 50: Command Runner

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
echo hello
free
relay status
help relay
```

■ What to check

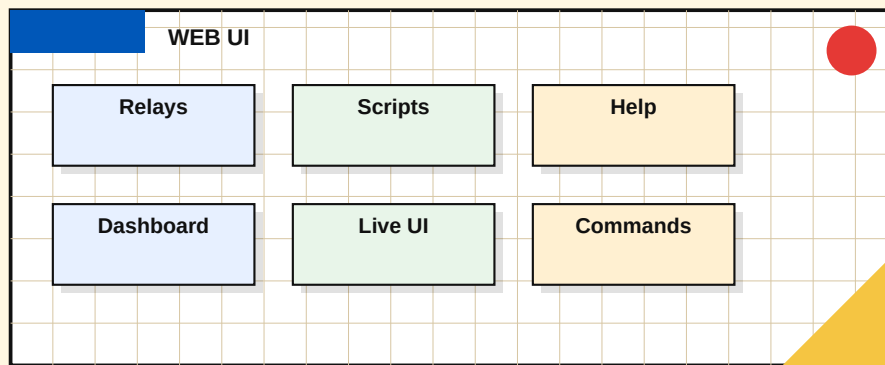
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Run the same command in serial and web and compare outputs.

51. Script Editor

The web editor makes scripts accessible without a serial terminal.



■ The mental model

It is a small file editor for /home and /etc scripts.

Why this matters

Validation catches simple mistakes before automation does.

▲ Common beginner mistake

Editing boot scripts directly without backup.

Lab 51: Script Editor

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
ls /home
cat /home/test.sh
sh -n /home/test.sh
```

What to check

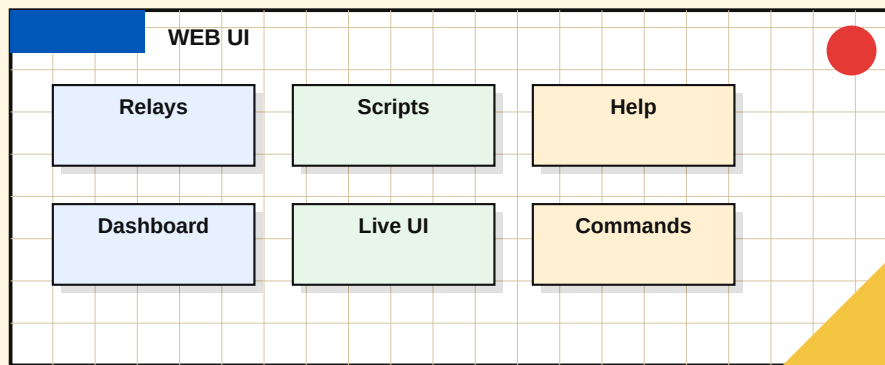
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Edit a script, validate, save, then run.

52. Local Help

Help files live on the board so documentation is available offline.



■ The mental model

The board carries its own pocket manual.

Why this matters

Offline help is valuable in gardens, workshops and basements.

▲ Common beginner mistake

Assuming online docs are required for every small task.

Lab 52: Local Help

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
help
help relay
ls /help
cat /help/scripts.txt
```

What to check

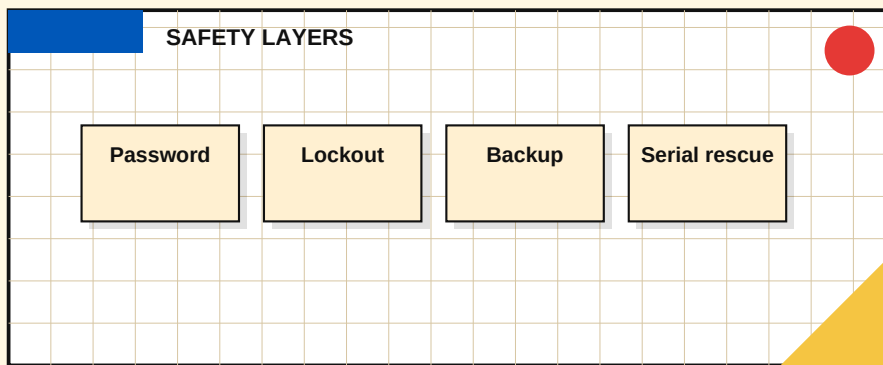
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Open the Help page in the web UI while disconnected from the internet.

53. Security Basics

Security starts with a good web key and careful network exposure.



■ The mental model

The board should be reachable by you, not by the whole world.

Why this matters

Garden automation can affect real hardware, so access matters.

▲ Common beginner mistake

Port-forwarding the ESP web UI to the internet.

Lab 53: Security Basics

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
config get web.key  
config set web.key <new-key>  
ap status  
wifi net
```

■ What to check

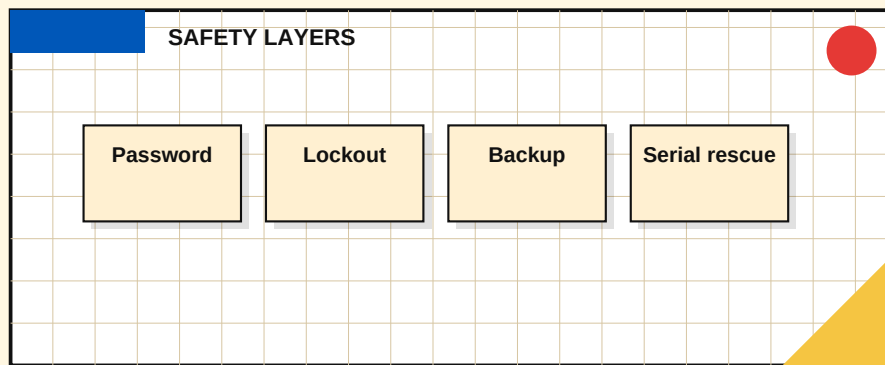
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Change the web key on a private bench network first.

54. Web Lockout

Lockout slows repeated bad password attempts.



■ The mental model

It is a seatbelt, not a castle wall.

Why this matters

Even small devices benefit from friction against guessing.

▲ Common beginner mistake

Depending on lockout while using a weak password.

Lab 54: Web Lockout

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
config get web.lockout
health
diag
```

What to check

- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Confirm lockout settings before field installation.

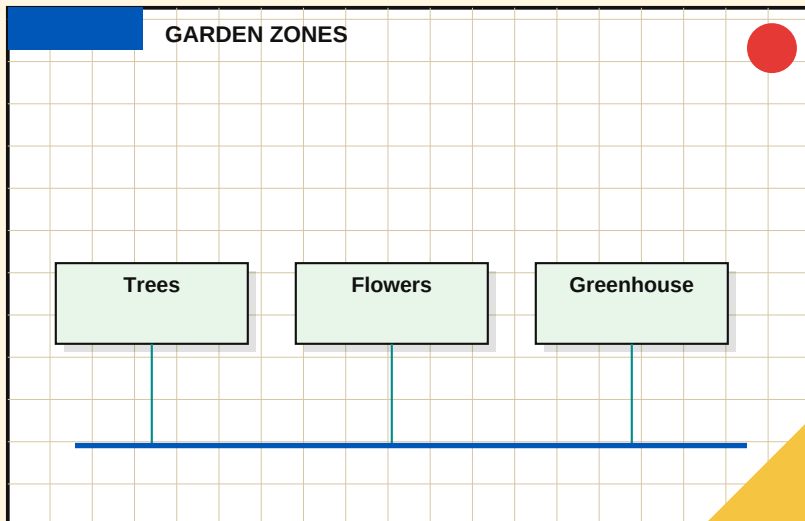
PART 8

Garden Projects

This part covers GPIO Choices, External Relay Power, Long Cable Runs and builds toward practical confidence.

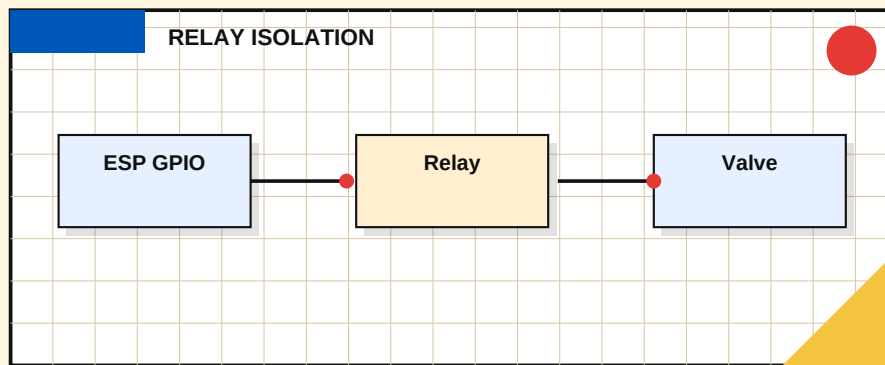
Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.



55. GPIO Choices

Not every GPIO is equally safe for relays and sensors.



■ The mental model

Some pins affect boot mode; choose conservative pins first.

Why this matters

Correct pin choice prevents boot surprises.

▲ Common beginner mistake

Using boot-sensitive pins without understanding pullups.

Lab 55: GPIO Choices

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
board pins
board show
pin D1
pin D2
```

What to check

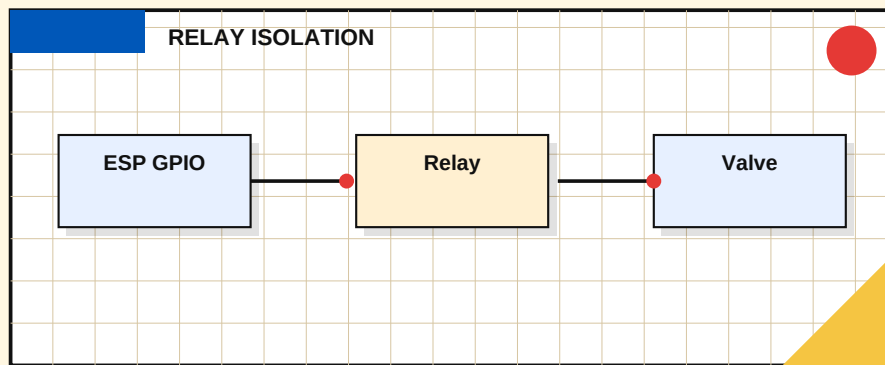
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Map every wire to a named relay or input.

56. External Relay Power

Relay modules should usually have their own suitable power supply.



■ The mental model

The ESP sends logic; the external supply does the coil work.

Why this matters

Stable power is more important than elegant code.

▲ Common beginner mistake

Powering many relay coils from the ESP regulator.

Lab 56: External Relay Power

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
relay status  
free  
health
```

What to check

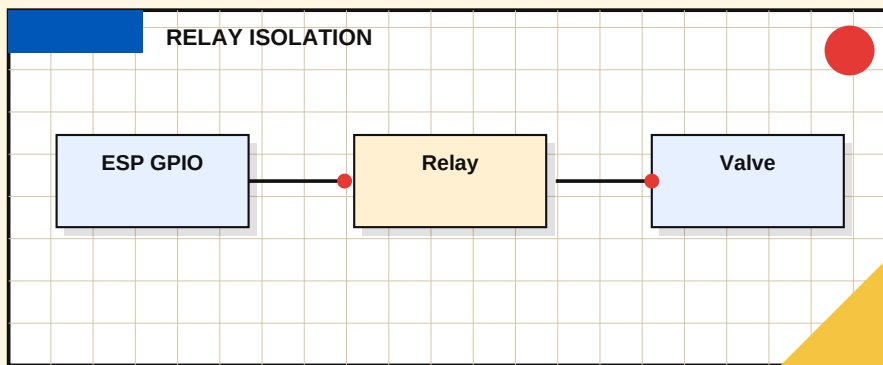
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Measure relay supply voltage while toggling.

57. Long Cable Runs

Long cables need planning for voltage drop, noise and protection.



■ The mental model

A cable is part of the circuit, not just a line on a drawing.

Why this matters

Field wiring fails differently from bench wiring.

▲ Common beginner mistake

Assuming a 100 m cable behaves like a 10 cm jumper.

Lab 57: Long Cable Runs

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
logger cable_test_start
relay pulse valve1 500
logger cable_test_done
```

What to check

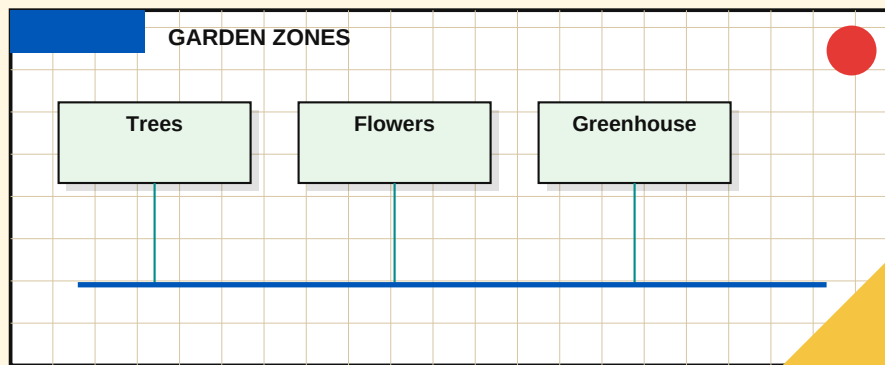
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Test with the actual cable length before burying anything.

58. Outdoor Enclosures

Outdoor electronics need protection from water, condensation, sun and frost.



■ The mental model

An enclosure is climate control for a tiny computer.

Why this matters

Mechanical protection is reliability work.

▲ Common beginner mistake

Putting a bare relay board in direct sun or frost.

Lab 58: Outdoor Enclosures

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
health
sensor read
log | tail -n 10
```

What to check

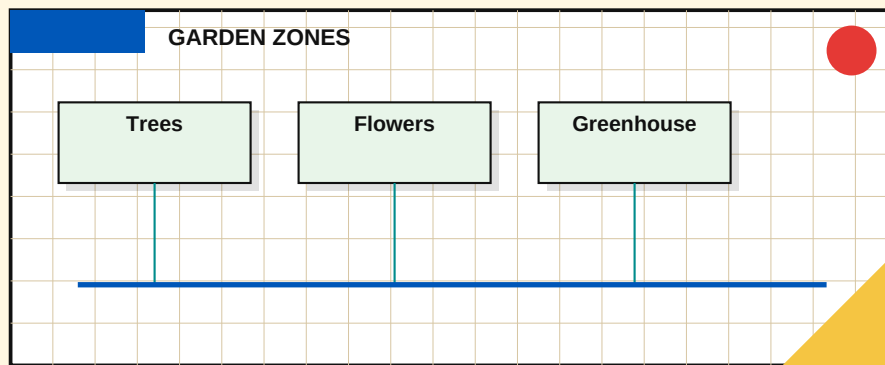
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Design for drip loops, strain relief and service access.

59. Solenoid Valves

Irrigation valves are usually better described as solenoids than pumps.



■ The mental model

The relay energizes a valve; water pressure does the work.

Why this matters

Solenoids make zone control practical.

▲ Common beginner mistake

Mixing AC and DC valve assumptions.

Lab 59: Solenoid Valves

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
relay add trees D1 active_low
relay on trees
timer once 900000 relay off trees
```

What to check

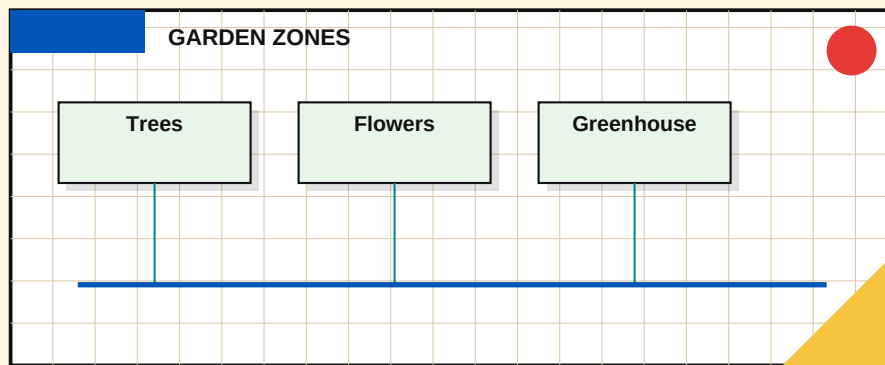
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Choose valve voltage before choosing the power supply.

60. Flow Measurement

A flow sensor can confirm that watering actually happened.



■ The mental model

Relay on means command sent; flow means water moved.

Why this matters

Feedback catches closed taps, clogged filters and broken wires.

▲ Common beginner mistake

Trusting relay state as proof of water delivery.

Lab 60: Flow Measurement

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
input add flow D7 pullup 20
input on flow change logger flow_pulse
log | tail -n 20
```

What to check

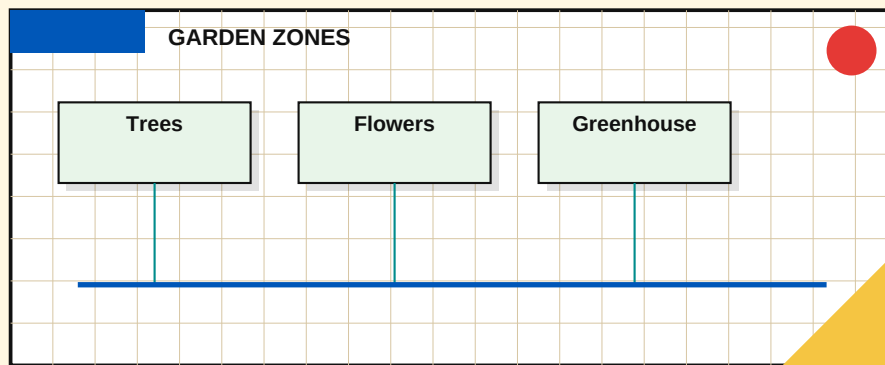
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Log pulses before converting them into liters.

61. Build Zone One

A first irrigation zone should be simple, observable and reversible.



■ The mental model

One zone is your learning sandbox.

Why this matters

A small first success makes the bigger system safer.

▲ Common beginner mistake

Building four zones before proving one zone.

Lab 61: Build Zone One

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
relay add trees D1 active_low
function water_trees { relay on trees; timer once 900000 relay off trees; logger trees_started
call water_trees
```

What to check

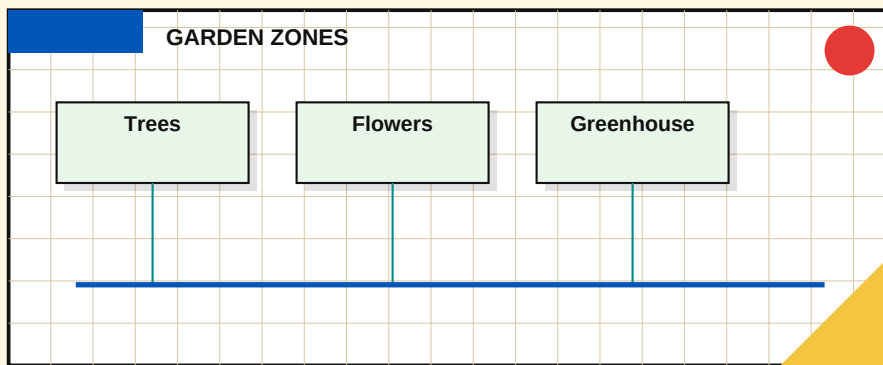
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Run the function with water off first.

62. Add Zone Two

Second zones teach naming, schedule spacing and power budgeting.



■ The mental model

Zones should not fight for water pressure or relay power.

Why this matters

Spacing zones is kinder to plumbing and power supplies.

▲ Common beginner mistake

Starting every valve at the same minute.

Lab 62: Add Zone Two

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
relay add flowers D2 active_low
function water_flowers { relay on flowers; timer once 300000 relay off flowers; logger flowers_
cron add daily 06:20 call water_flowers
```

What to check

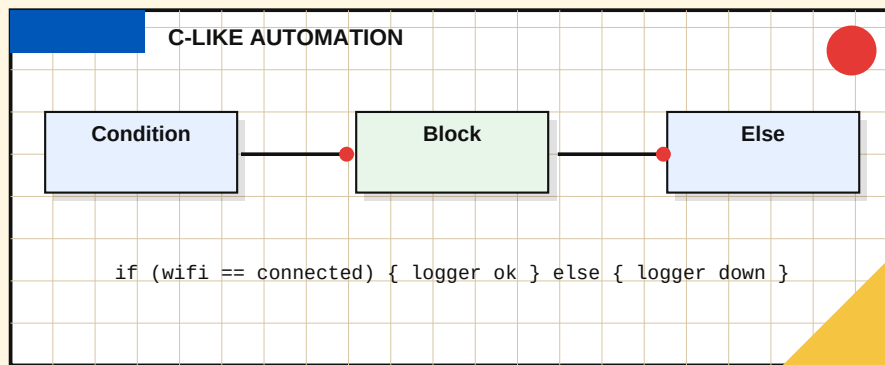
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Leave time between zones and observe pressure.

63. Rain Skip Pattern

A rain input can pause watering.



■ The mental model

A sensor changes a variable; schedules read that variable.

Why this matters

Separating sensing from watering keeps logic understandable.

▲ Common beginner mistake

Letting a noisy sensor directly toggle a valve.

Lab 63: Rain Skip Pattern

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
let irrigation.enabled = 1
input add rain D5 pullup 50
when input rain low let irrigation.enabled = 0
let list
```

What to check

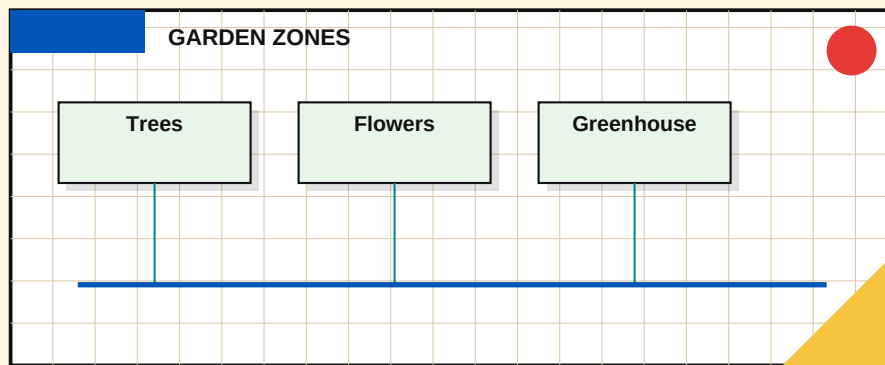
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Use logger first if your rain sensor logic is unknown.

64. Temperature Window Pattern

Watering can be restricted by temperature and time.



■ The mental model

Conditions protect plants and hardware.

Why this matters

A safe window reduces evaporation and heat stress.

▲ Common beginner mistake

Forgetting that time needs NTP.

Lab 64: Temperature Window Pattern

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
define MAX_TEMP 35
define MORNING_END 10:00
if (temp < MAX_TEMP && time < MORNING_END) { relay on trees; timer once 900000 relay off trees
```

What to check

- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

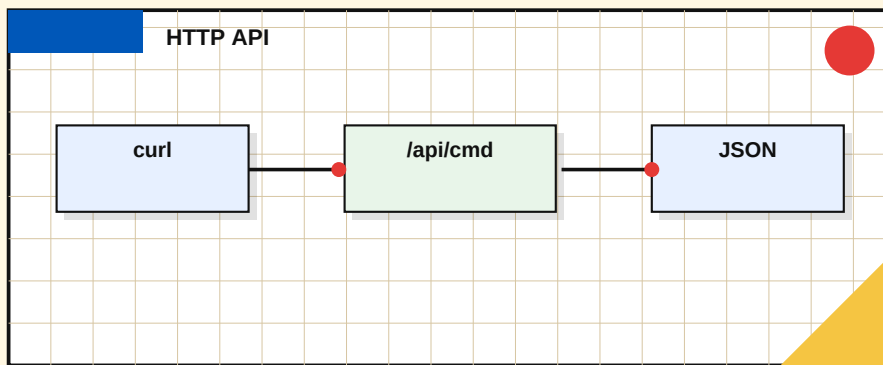
Try this next

Test the false branch with an impossible threshold.



65. Daily Health Report

A daily report proves the board is alive.



■ The mental model

Heartbeat messages are boring until they save a trip.

Why this matters

Health reports catch silent failures.

▲ Common beginner mistake

Sending reports too often and training yourself to ignore them.

Lab 65: Daily Health Report

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
function daily_health { if (wifi == connected) mail health "KernelESP daily health" else logger  
cron add daily 08:00 call daily_health
```

What to check

- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Start with logger if mail is not configured.

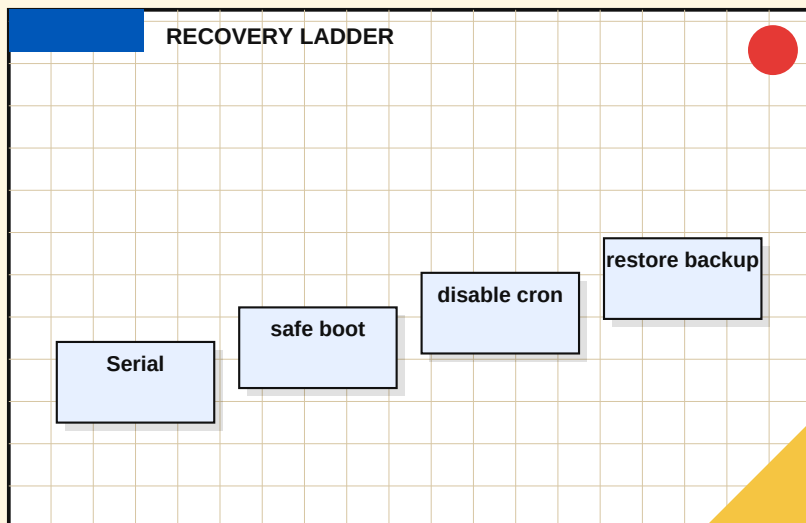
PART 9

Recovery and Release

This part covers Recover from Bad Automation, Serial Rescue Workflow, Safe Boot Thinking and builds toward practical confidence.

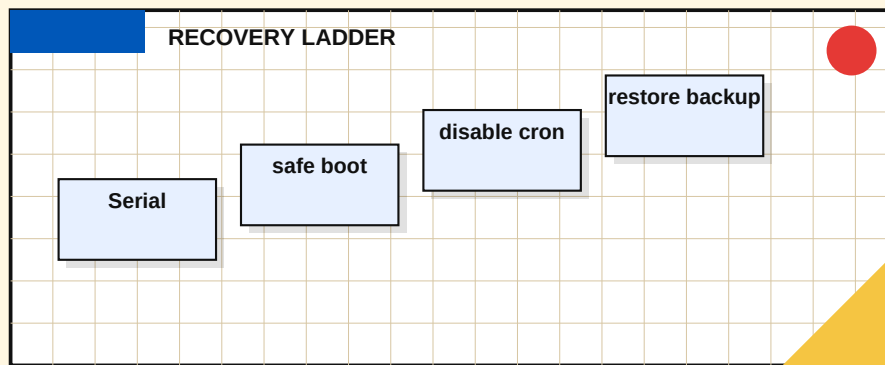
Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.



66. Recover from Bad Automation

Bad cron or scripts should be removable without reflashing.



■ The mental model

Recovery means pause, inspect, remove, test.

Why this matters

A practiced recovery path makes experimentation safer.

▲ Common beginner mistake

Trying random fixes while automations are still armed.

Lab 66: Recover from Bad Automation

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
disarm
crontab -l
cron rm <id>
timer clear
arm
```

What to check

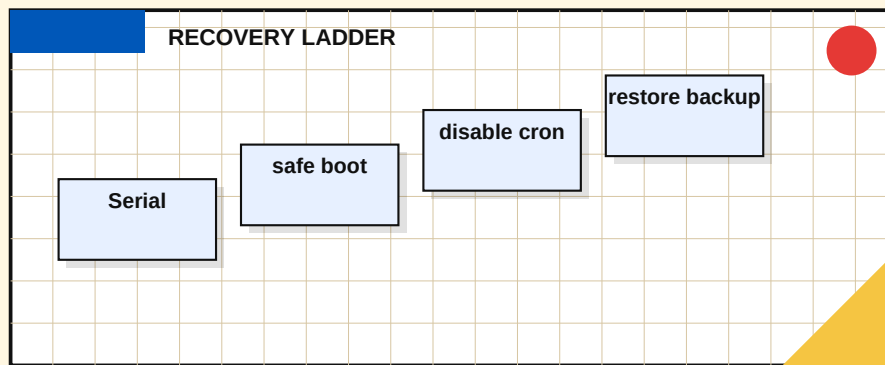
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Practice disarming before you need it.

67. Serial Rescue Workflow

Serial rescue is the final trusted path when web access is slow or gone.



■ The mental model

When the network is uncertain, go local.

Why this matters

Many apparent firmware failures are network or script mistakes.

▲ Common beginner mistake

Reflashing before reading logs.

Lab 67: Serial Rescue Workflow

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
free
wifi status
crontab -l
input list
reboot
```

What to check

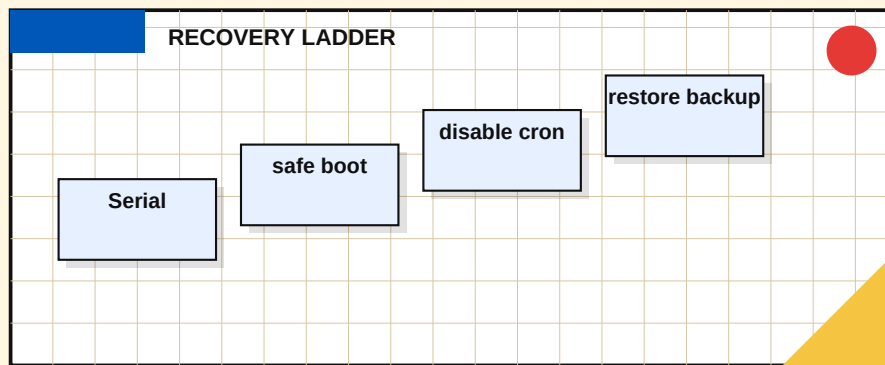
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Keep a known-good cable and serial command list nearby.

68. Safe Boot Thinking

Safe boot keeps startup from becoming a trap.



■ The mental model

Boot should prepare the system, not perform risky work blindly.

Why this matters

Power failures happen; startup behavior must be predictable.

▲ Common beginner mistake

Starting pumps automatically at boot without checks.

Lab 68: Safe Boot Thinking

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
boot show
cat /etc/boot.sh
safe status
dmesg
```

What to check

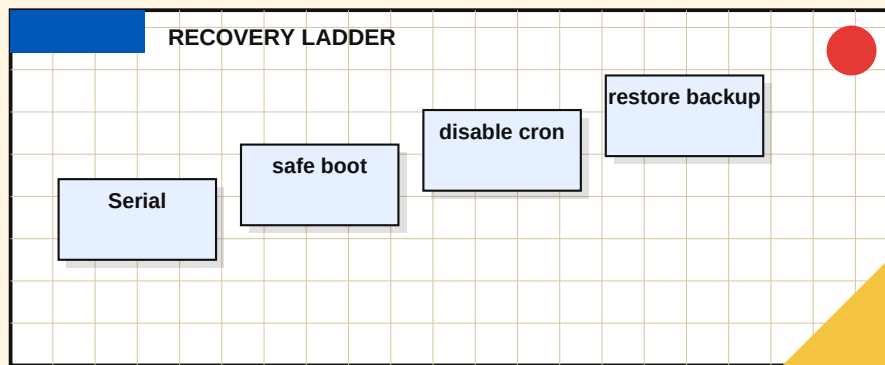
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Review boot commands after every major automation change.

69. Firmware Upload

Uploading firmware updates the program while preserving a disciplined workflow.



■ The mental model

Firmware is the operating brain; assets and files are separate concerns.

Why this matters

A repeatable upload flow avoids mystery states.

▲ Common beginner mistake

Uploading firmware and forgetting web assets.

Lab 69: Firmware Upload

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
tools/verify.sh
tools/upload.sh /dev/cu.usbserial-XXXX
tools/smoke-http.sh http://<ip> admin
```

What to check

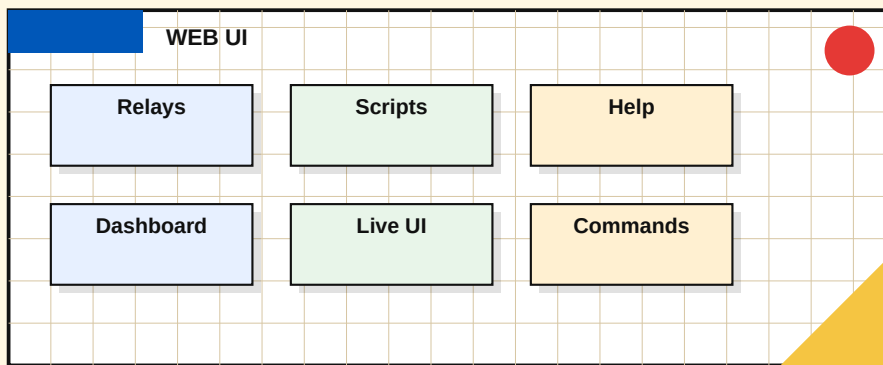
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Never skip verification on a release candidate.

70. Asset Upload

Web assets update the browser interface and help files.



■ The mental model

The firmware serves files from /www and /help.

Why this matters

UI fixes may live in files, not firmware.

▲ Common beginner mistake

Assuming a firmware flash updates every web file.

Lab 70: Asset Upload

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
tools/upload-assets.sh http://<ip> admin  
ls /www  
ls /help
```

What to check

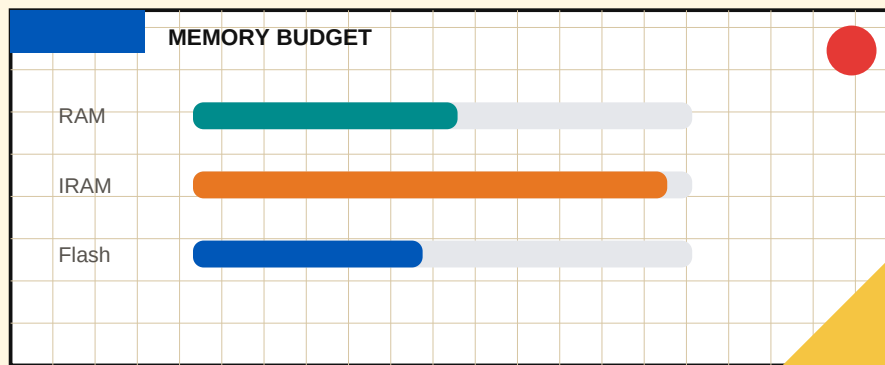
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

After asset upload, refresh the browser hard.

71. Release Testing

A release is not done until checks pass.



■ The mental model

Testing is a ritual that protects future users.

Why this matters

Small devices need boring repeatability.

▲ Common beginner mistake

Calling it done after only compiling.

Lab 71: Release Testing

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
tools/verify.sh
tools/smoke-http.sh http://<ip> admin
COUNT=60 DELAY=1 tools/stability-http.sh http://<ip> admin
```

What to check

- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Keep test output with your release notes.

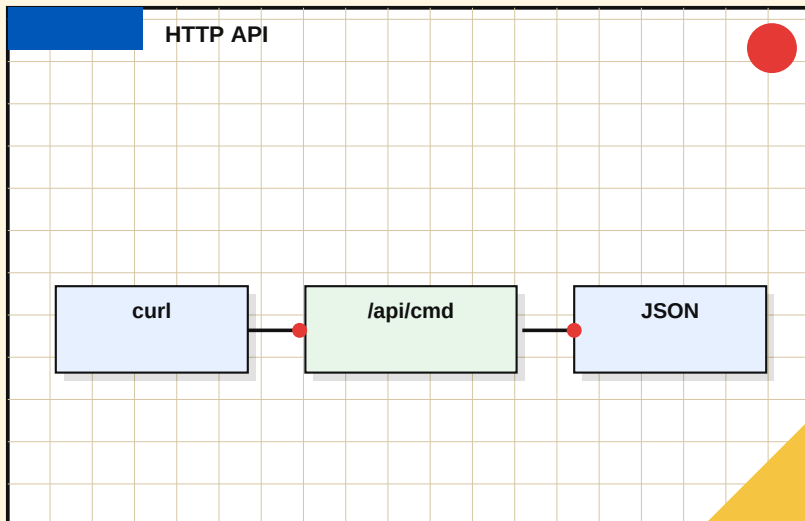
PART 10

Going Further

This part covers Command Reference Habits, API Integrations, Build a Morning Irrigation Program and builds toward practical confidence.

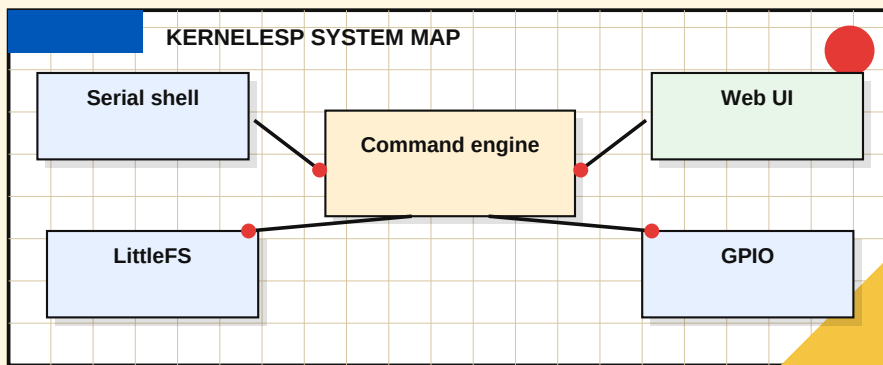
Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.



72. Command Reference Habits

The command reference is easier when you read by task, not alphabetically.



■ The mental model

Ask: what am I trying to inspect or change?

Why this matters

Beginners do not need every command at once.

▲ Common beginner mistake

Trying to memorize the whole shell.

Lab 72: Command Reference Habits

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
help
help cron
help relay
help scripts
```

What to check

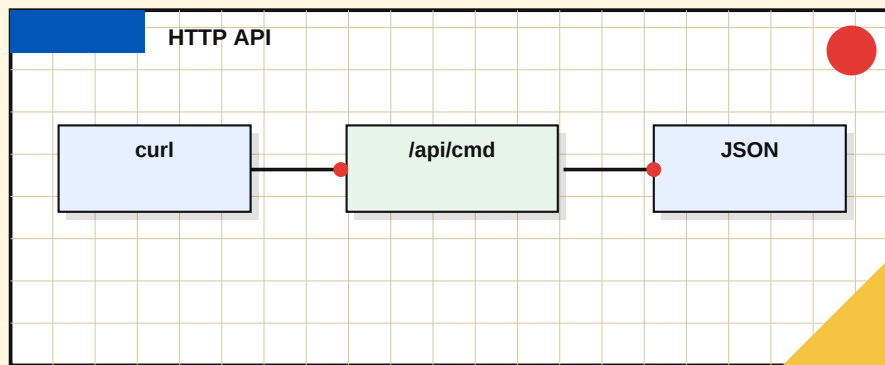
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Make your own cheat sheet of ten commands.

73. API Integrations

External systems can call KernelESP without replacing its local control.



■ The mental model

The API is a bridge, not the boss.

Why this matters

Read-only integration is the safest first step.

▲ Common beginner mistake

Letting a remote system send actuator commands before local safety exists.

Lab 73: API Integrations

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
curl -G http://<ip>/api/cmd --data-urlencode key=admin --data-urlencode c='relay status'
curl http://<ip>/api/status?key=admin
```

What to check

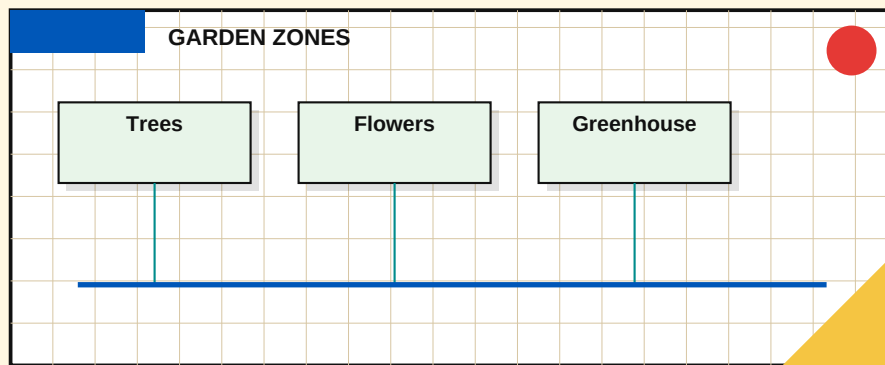
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Write one laptop script that reads status only.

74. Build a Morning Irrigation Program

A complete morning program combines time, variables, relays and timers.



■ The mental model

The readable command is the design.

Why this matters

Manual first, cron second is the safest habit.

▲ Common beginner mistake

Scheduling a command you have never run manually.

Lab 74: Build a Morning Irrigation Program

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
let irrigation.enabled = 1
define WATER_END 10:00
if (irrigation.enabled == 1 && time < WATER_END) { relay on trees; timer once 900000 relay off
```

What to check

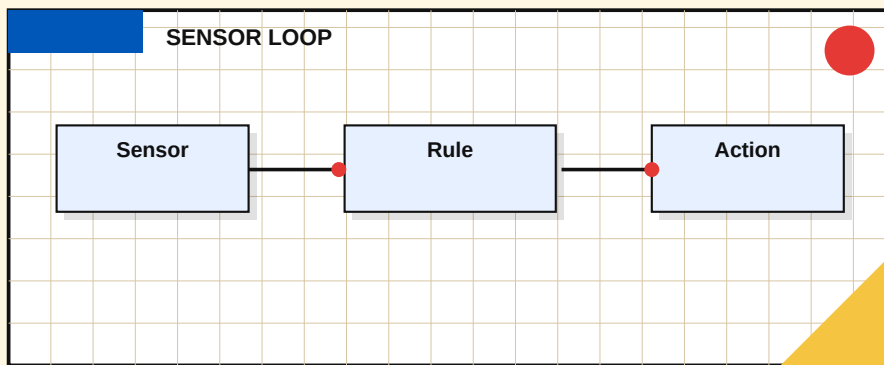
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Run it manually before scheduling it.

75. Build a Temperature Fan Program

Temperature control is a classic automation pattern.



■ The mental model

Sense, decide, act, log.

Why this matters

The same pattern works for heaters, extractors and alarms.

▲ Common beginner mistake

Skipping the off condition.

Lab 75: Build a Temperature Fan Program

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
define HOT 40
define COOL 35
if (temp >= HOT) { relay on fan; logger fan_on }
if (temp <= COOL) { relay off fan; logger fan_off }
```

What to check

- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Use a lamp or LED before a real fan.

76. Build an Offline Logger

When Wi-Fi is down, logging still gives evidence.

LITTLEFS LAYOUT	
/	
/etc	config, cron, relays
/home	scripts and notes
/www	web interface
/var/log	kernel.log
/func	persistent functions

■ The mental model

Local logs are the fallback memory.

Why this matters

Good automations degrade gracefully.

▲ Common beginner mistake

Assuming every alert can use the network.

Lab 76: Build an Offline Logger

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
if (wifi == connected) { mail health "online" } else { logger offline_no_mail }  
tail -n 20 /var/log/kernel.log
```

What to check

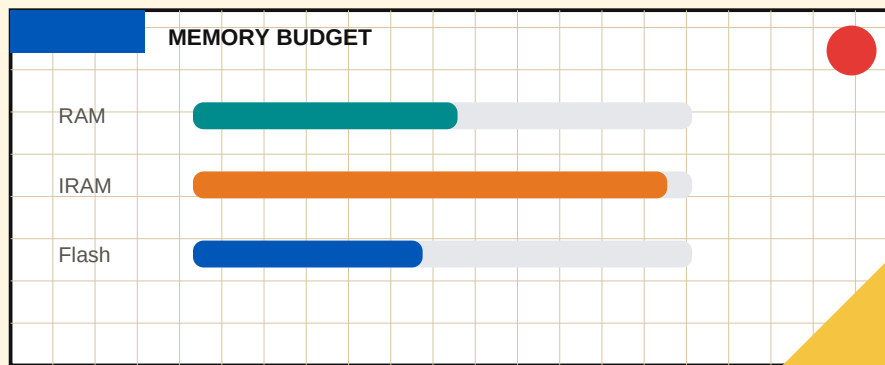
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Force a false condition and confirm the local log path.

77. Keep the System Fast

The ESP8266 web server is small; fewer refreshes and short commands help.



■ The mental model

Responsiveness is a shared resource.

Why this matters

A fast UI is often a quiet UI.

▲ Common beginner mistake

Polling many expensive pages every second.

Lab 77: Keep the System Fast

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
free
health
ps
top
```

■ What to check

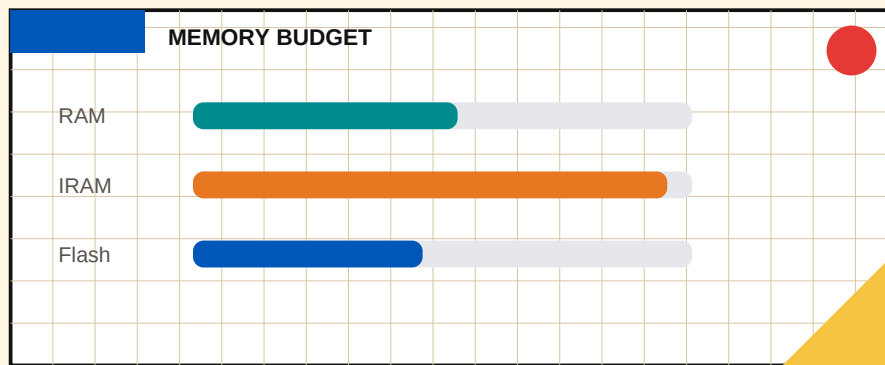
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Close extra Live UI tabs and compare heap/latency.

78. Know the Limits

KernelESP is powerful because it is small, not because it is unlimited.



■ The mental model

Limits are design rails.

Why this matters

Knowing limits prevents accidental complexity.

▲ Common beginner mistake

Treating ESP8266 RAM like a desktop computer.

Lab 78: Know the Limits

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
free
df
timer list
crontab -l
input list
```

■ What to check

- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Write down your board's active relays, timers, inputs and cron jobs.

79. Document Your Installation

A garden controller needs notes as much as code.

LITTLEFS LAYOUT	
/	
/etc	config, cron, relays
/home	scripts and notes
/www	web interface
/var/log	kernel.log
/func	persistent functions

■ The mental model

Documentation is part of the system.

Why this matters

Future maintenance starts with knowing what exists.

▲ Common beginner mistake

Relying on memory for outdoor wiring.

Lab 79: Document Your Installation

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
write /home/INSTALL.txt valve1=trees
append /home/INSTALL.txt D1=trees relay
cat /home/INSTALL.txt
```

What to check

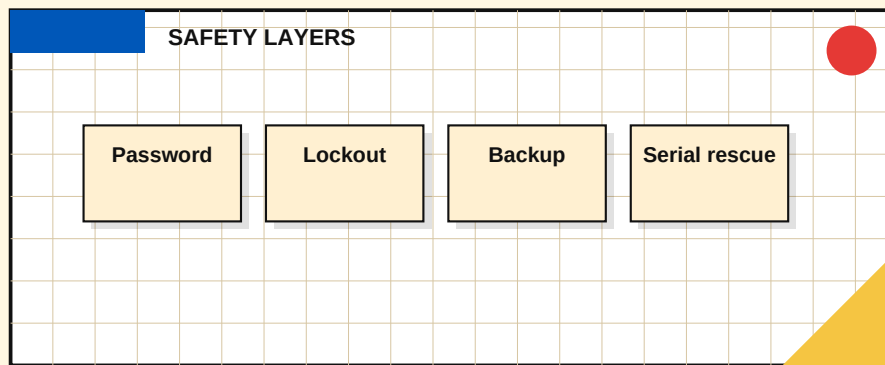
- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Store pin and valve notes on the board and in your repo.

80. Final Beginner Checklist

A safe project is inspected, backed up, tested and recoverable.



■ The mental model

The checklist is your calm co-pilot.

Why this matters

The best automation is one you can explain and recover.

▲ Common beginner mistake

Leaving a new automation alone without observing one full cycle.

Lab 80: Final Beginner Checklist

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
health
free
df
crontab -l
timer list
input list
backup
```

■ What to check

- The command should return promptly. If it does not, stop adding new automations and check Wi-Fi, heap and logs.
- The output should tell you what changed. KernelESP is designed to be inspectable, so verify state after every important action.
- If the command controls hardware, test first with dryrun or without mains voltage connected.

Try this next

Run the checklist before leaving the installation unattended.

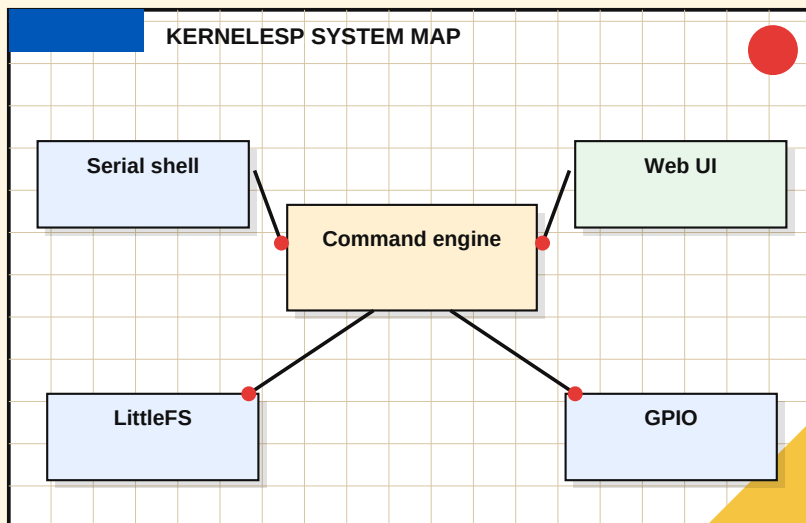
PART 11

Command Atlas

A compact appendix for commands you will use again and again.

Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.





Shell Essentials

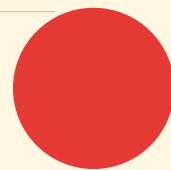


These are the commands you use while learning and debugging. They make the board feel like a tiny UNIX shell.

help	man
clear	echo
history	alias
unalias	env
printenv	set
unset	true
false	test
repeat	watch



System Information

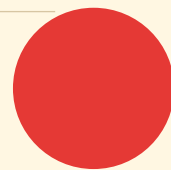


These commands answer the beginner question: is the board healthy, busy or running out of resources?

version	uname
uptime	free
heap	mem
ps	top
pgrep	pidof
kill	dmesg
reboot	resetreason
chip	flash
sysinfo	health
diag	



Files and Storage

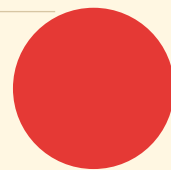


LittleFS is small, so inspect files and space regularly.

<code>pwd</code>	<code>cd</code>
<code>ls</code>	<code>cat</code>
<code>head</code>	<code>tail</code>
<code>grep</code>	<code>find</code>
<code>wc</code>	<code>du</code>
<code>stat</code>	<code>touch</code>
<code>write</code>	<code>append</code>
<code>rm</code>	<code>mkdir</code>
<code>rmdir</code>	<code>cp</code>
<code>mv</code>	<code>df</code>
<code>fsformat</code>	



Network and Time



Networking and time are the foundation for web access, API integrations, logs and cron.

wifi scan

wifi net

wifi connect

wifi reconnect

wifi recover

ifconfig

ap status

date

ntp sync

wifi status

wifi diag

wifi save

wifi wait

wifi sdkreset --yes

ip addr

hostname

ntp kick



Automation and Hardware



These are the commands that turn KernelESP from a tiny computer into a controller.

```
relay add
relay off
timer once
cron add
rule add
state set
define
call
when input
mail health
```

```
relay on
relay pulse
timer every
crontab -l
scene add
let
function
input add
sensor begin
```

Final Notes

KernelESP is deliberately small. That is its charm and its discipline. The board will reward careful habits: name things well, keep actions short, use timers, log important decisions, back up before experiments and keep a serial rescue path close.

The beginner's rule

If you can inspect it, explain it and undo it, you are ready to automate it.

Before leaving a board unattended

```
health
free
df
crontab -l
timer list
input list
backup
```